An Empirical Study on Challenges of Event Management in Microservice Architectures

RODRIGO LAIGNER, Department of Computer Science, University of Copenhagen, Copenhagen, Denmark
ANA CAROLINA ALMEIDA, IME, Rio de Janeiro State University, Rio de Janeiro, Brazil
WESLEY K. G. ASSUNÇÃO, Computer Science, North Carolina State University at Raleigh, Raleigh, North Carolina, USA
YONGLUAN ZHOU, Department of Computer Science, University of Copenhagen, Copenhagen, Denmark

Microservices emerged as a popular architectural style over the last decade. Although designed to be self-contained, microservices must communicate to realize business capabilities, creating dependencies among their data and functionalities. Developers then resort to asynchronous, event-based communication to fulfill such dependencies while reducing coupling. However, developers are often oblivious to the inherent challenges of the event-based communication paradigm, leading to pitfalls that ultimately prompt them to reconsider adopting microservices. To make matters worse, the literature is scarce on the practices and challenges of designing, implementing, testing, monitoring, and troubleshooting event-based microservices.

To fill this gap, this article provides the first comprehensive characterization of event management practices and challenges in microservices, based on a repository-mining study of over 8,000 Stack Overflow questions. We find that developers encounter a gamut of problems, including handling large event payloads, modeling event schemas, auditing and debugging event flows, and managing event dependencies and ordering constraints, suggesting that current practices and technologies do not sufficiently serve developers. We review the state of the art and practice through the lens of unveiled challenges and provide actionable implications for developers, providers, and researchers. As a result, our study will support advancing event management solutions in microservices.

CCS Concepts: • **Computer systems organization** → **Distributed architectures**; • **Software and its engineering**;

Additional Key Words and Phrases: microservice, asynchronous, event, streams, pubsub, decoupling, event-driven architecture

This work was supported by the PAPRICAS.org project—Independent Research Fund of Denmark (Number 9131-00077B). Work by Ana Almeida was done while employed as a postdoc at the University of Copenhagen.
Authors' Contact Information: Rodrigo Laigner (corresponding author), Department of Computer Science, University of Copenhagen, Copenhagen, Denmark; e-mail: rnl@di.ku.dk; Ana Carolina Almeida, IME, Rio de Janeiro State University, Rio de Janeiro, Brazil; e-mail: ana.almeida@ime.uerj.br; Wesley K. G. Assunção, Computer Science, North Carolina State University at Raleigh, Raleigh, North Carolina, USA; e-mail: wguezas@ncsu.edu; Yongluan Zhou, Department of Computer Science, University of Copenhagen, Copenhagen, Denmark; e-mail: zhou@di.ku.dk.



This work is licensed under Creative Commons Attribution International 4.0.

© 2026 Copyright held by the owner/author(s).
ACM 1557-7392/2026/7-ART245
https://doi.org/10.1145/3776581

ACM Reference format:
Rodrigo Laigner, Ana Carolina Almeida, Wesley K. G. Assunção, and Yongluan Zhou. 2026. An Empirical Study on Challenges of Event Management in Microservice Architectures. *ACM Trans. Softw. Eng. Methodol.* 35, 8, Article 245 (July 2026), 62 pages.
https://doi.org/10.1145/3776581

1 Introduction

The emergence of cloud computing as a paradigm for large-scale deployment of services has prompted industry practitioners to rethink how applications are architected and deployed [6]. In particular, we witness a growing adoption of *microservice architectures* [222]. Microservice architectures promote designing components as independent building blocks that are deployed independently and interact with each other via network protocols [74]. This architectural style brings more flexibility for the development and deployment of large systems, in contrast to conventional architectures, in which software components are strongly coupled with each other and deployed as a whole unit [140].

The main benefit of microservices is that development teams can develop and operate their services independently [210]. Microservices should be designed as self-contained software units that operate autonomously [140]. To realize business capabilities, microservices need to communicate, which ends up creating dependencies among their data and functionalities [115]. To manage such dependencies while maximizing decoupling, developers often rely on asynchronous event-based communication between microservices, instead of using traditional remote procedure calls [110, 114, 147]. The rationale is that, through events, producers and consumers are not directly coupled. For instance, producer microservices can trigger operations in other microservices or communicate their own state updates through events. Such an approach may positively influence the application's ability to evolve over time, limit the propagation of faults, and favor increased scalability of individual components [110, 154].

Event-Driven Architecture (EDA) has been rapidly gaining industry popularity [25]. However, despite the benefits of an event-driven design, developers frequently report challenges in managing events in industrial settings [32, 59, 81, 114, 115, 142, 143, 147, 175, 176, 179, 180, 217]. Not surprisingly, it is common to encounter developers seeking support on how to implement event-driven microservices appropriately, such as this comment found on **Stack Overflow (SO)** (a popular Questions and Answers forum) [40]: "*Implementing an eventually consistent distributed architecture has turned out to be a pain. There are tons of blog posts telling stories about how to do it, but not showing (code) how to actually do it. One of the aspects I'm suffering is having to deal with manual retries of the messages when they haven't been acknowledged.*" To make matters worse, in our study, we observed that there is a scarcity of literature on the practices and challenges that the adoption of an asynchronous paradigm brings to the designing, implementing, testing, monitoring, and troubleshooting of microservices. The challenges of managing communication in asynchronous and event-based systems have been explored in the fields of databases and distributed systems [28, 202], but this knowledge is not widely disseminated among developers [23, 115], making them sometimes try to "reinvent the wheel" when addressing common challenges. This limited understanding of existing practices and how to address challenges in architectures based on asynchronous communication leads to frustrations and ultimately makes companies reconsider the adoption of microservices [83, 129, 188].

There has been extensive work investigating the adoption and implications of microservice architectures [15, 57, 115, 130, 138, 162, 201, 208, 209, 219, 221]. However, these pieces of work mostly focus on **Representational State Transfer (REST)** architectural style, describe the benefits

of using microservices, or characterize data management and security practices without a holistic view of the practices and challenges. Furthermore, studies target specific companies or a limited number of practitioners, reducing the generalizability of their findings. To the best of our knowledge, no work has systematically investigated the practices and challenges brought about by an *event-driven* design in microservice architectures. As a result, although *event-driven* microservices form an important portion of microservice deployments today [115, 209], it is unclear what particular challenges developers face when developing and operating them. The need to address these issues is even more relevant when we consider studies suggesting there is a tension between the use of asynchronous microservice designs and application safety [115].

This work aims to characterize the practices and challenges faced by developers when adopting event-based microservice architectures. More specifically, we first investigate the practices adopted by developers to manage events in microservice architectures. We aim to comprehend current technological trends and common implementation patterns related to the adoption of event management. We also seek to identify the functional requirement and **Non-Functional Requirement (NFR)** that are met or supported by implementing event management in microservices. Secondly, we want to identify recurrent challenges faced by developers when managing events in microservice architectures. Focusing on these two perspectives, we can equip developers with knowledge of existing practices for event management in microservices and make them aware of the challenges they will face when developing asynchronous and event-based microservices.

To achieve our goal, we perform, to the best of our knowledge, the first empirical study to comprehend the practices and identify the challenges in managing events in microservice architectures based on discussions of developers. To this end, our study is based on repository mining [82]. We mine and analyze relevant questions from SO [185], one of the most popular Questions and Answers forums for developers who seek technical advice or assistance [31]. We collect more than 8,000 SO questions associated with managing events in microservice architectures. Via a mix of keyword analysis and manual examination of tags, questions, and related responses (i.e., posts), we collect the most popular patterns and NFR and functional requirement mentioned in the context of event management in microservices. Then, we randomly sample 628 relevant SO questions for manual analysis to identify the challenges. We carefully extract the challenges behind each question through a peer-reviewed process. We classify the uncovered challenges into different properties of distributed systems and functional requirement and NFR. These ultimately represent key areas that microservice developers struggle with.

In the quantitative analysis, we observe that event management has been gaining increasing attention in practice, indicating the emergence of this architectural paradigm and the timeliness of this study. Results show the popularity of patterns and requirements related to achieving data consistency and loose coupling. This suggests microservice developers seek to strike a better balance between consistency and decoupling in their microservice architectures by employing asynchronous events, especially in computations that span microservices. Furthermore, we observe that developers report different categories of patterns in their implementations. For instance, observability (e.g., *distributed tracing*), performance (e.g., *circuit breaker*), and security (e.g., *access token*) patterns are reported to be applied in questions involving event management.

In the qualitative analysis, we find that microservice practitioners face a myriad of challenges in managing events. While events are supposed to maximize the decoupling of microservices [69, 114], developers often surprisingly necessitate synchronizing events for correct event processing and coordinating microservices to allow for software evolution, contradicting the alleged benefits of an event-based architecture [72, 154]. Besides, although events can serve as natural progress markers of microservices [72, 110], we observe that developers find little benefit in using events to monitor and troubleshoot microservices. In particular, developers have a hard time tracking down the result of

their computations, which often span a network of dependent microservices and exhibit additional difficulties in replaying past events for debugging purposes. Safeguarding security properties is also an emerging challenge in event-based microservice architectures. Developers struggle to synchronize event management and security technologies to ensure that only events generated by secured channels are processed and that only authenticated microservices can consume them.

This article's contributions are manifold. We evidence that microservice developers are insufficiently served by state-of-the-practice technologies, including messaging systems, frameworks, databases, and cloud providers. They end up implementing their own ad hoc solutions to fulfill their requirements, which ultimately leads to errors and bugs, rendering adoption of a microservice architecture frustrating. Characterizing this emerging practice well informs general software developers, especially those unfamiliar with the challenges of designing distributed applications, about the hidden dangers of asynchronous, event-based microservice designs. We derive actionable implications for each class of event management challenges to researchers and message, database, cloud, and framework providers. These relate but are not limited to, devising novel microservice programming models, event management architectures, tools, and methods to advance the state-of-the-art and practice in managing events in microservice architectures, making this a timely and valuable study. Ultimately, the dataset used in this study is made available[1] as an additional contribution to the research community, allowing other researchers to further investigate the theme.

The remainder of this work is organized as follows: Section 2 describes the role of events in modern microservice architectures and typical issues developers encounter. In Section 3, we present the methodology employed in this work. Section 4 presents the state of practice. Section 5 presents the specific challenges microservice developers encounter in managing events, discusses the existing practices and tools for minimizing the impact and mitigating some of them and the implications for the research community and industry providers. Section 6 discusses the threats to validity. Section 7 presents the related work and an overview of research lines that intersect with some of the goals of microservice architectures, highlighting their shortcomings in relation to the challenges unveiled in our study. Section 8 concludes this work.

2 Background and Motivation

2.1 Microservice Architectures

The *microservice architectural style* arose as a popular strategy for building applications in the cloud [222]. In contrast to the conventional monolithic architecture, where application components are strongly coupled and communicate with each other via function calls under a single execution unit, a microservice architecture promotes high modularity, designing applications as independent components that interact with each other via mechanisms such as HTTP-based protocols or asynchronous messages [140].

The decoupled design promoted by microservice architectures exhibits both advantages and disadvantages. On the one hand, microservices can be polyglot, using different programming languages and independent databases [115]. Besides, microservices promote fault isolation, where faults can be constrained to an individual microservice boundary, e.g., through a container-based deployment. The latter refrains failures from propagating to other components of the architecture [71].

2.2 Event Management

Despite being designed to operate autonomously, microservices often show data and functionality dependencies [115]. To manage these dependencies while maximizing decoupling, practitioners

[1] https://zenodo.org/records/13149520.

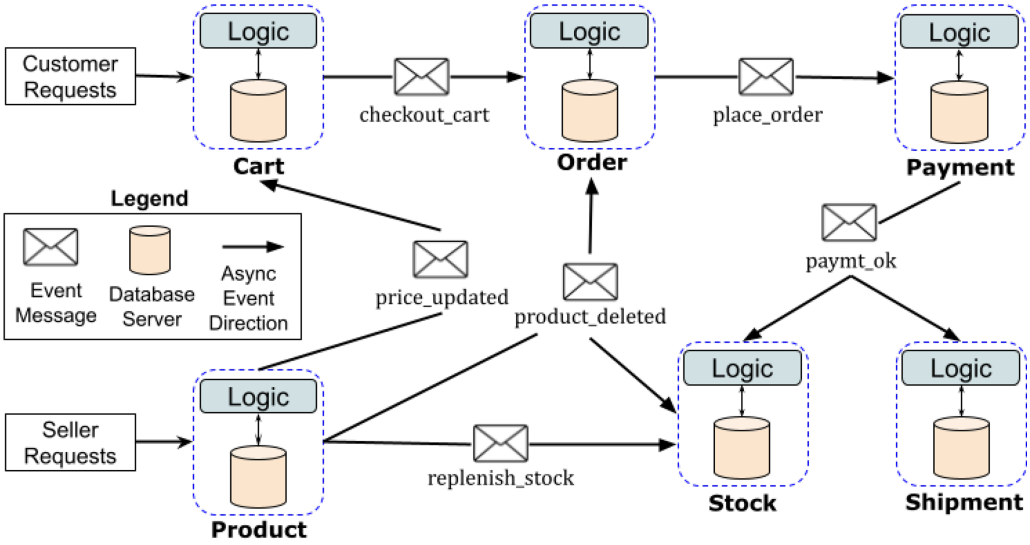


Fig. 1. Events in a microservice architecture.

increasingly rely on asynchronous, event-based communication between microservices instead of synchronous communication (e.g., HTTP-based protocols) [115], as noted by Netflix [172]:

> "Rethinking service interactions as streams of event exchanges (as opposed to a sequence of synchronous requests) lets us build infrastructure that is inherently asynchronous. It promotes decoupling and provides traceability as a first-class citizen in a web of distributed transactions."

An event serves as a logical unit of communication across microservices. It often carries a payload containing useful semantic data about the behavior of the producer microservice (e.g., invoice issued) or the expected outcome of the consumer microservices (e.g., process payment) [69]. Besides communicating actions, events are leveraged by large-scale architectures to aid state management, as noted below:

> "Events are much more than triggers and updates. They become the immutable stream from which we can reconstruct the entire system state. Moving towards a publish/subscribe model enables every service to publish its changes as events into a message bus, which can then be consumed by another service of interest that needs to adjust its state of the world" [172].

To illustrate the role events play in microservice architectures, we use Figure 1 to illustrate an e-commerce platform. We get inspiration from a popular microservice open source project [5] that incorporates real-world event management patterns. The application is composed of the following microservices: *Cart*, entailed for keeping track of user's cart items, *Stock*, responsible for managing the availability of products; *Order* responsible for the processing of orders, *Payment* processes the payment of order invoices, and *Product*, that maintains products' information. The microservices rely on a message broker to communicate with each other through asynchronous events (represented with a letter figure). The events cut across several microservices and serve as the building blocks for composing the functionalities of different microservices.

Upon a customer's checkout request, the *Cart* microservice spawns the processing of a checkout by generating a `checkout_cart` event. The event is then consumed by the *Order* microservice, which processes the cart items and generates the `place_order` event. Next, the *Payment* microservice is

responsible for processing the payment and generating the `paymt_ok` for downstream consumption by the *Stock* and *Shipment* microservices, which can process the event independently.

Besides customer checkouts, sellers may be willing to adjust product prices, deprecate some products, or replenish stock. In our example, *Product* acts as the user-facing microservice for processing seller requests. In the first case, the *Product* microservice updates the price accordingly and generates a `price_updated` event so the *Cart* can update its cache. In the second case, upon discontinuing the product, the *Product* microservice generates a `product_deleted` event for both *Order* and *Stock* microservices, so they can react accordingly to the new fact. For last, the *Product* microservice generates a `replenish_stock` event for the *Stock* microservice.

It is noteworthy that, in synchronous communication paradigms, such as through REST APIs and RPCs, the requester microservice gets blocked until the request terminates in the remote microservice and the response is received. It is natural to deduce that such blocking times can grow arbitrarily when business transactions traverse several microservices. In the example above, *Cart* would necessarily wait until *Payment* terminates the payment processing. Through the asynchronous paradigm, though, once the `checkout_cart` event is generated, *Cart* microservice is free to allocate computational resources (e.g., threads, CPUs, and memory) to serve other requests. This usually favors higher efficiency in event processing [139].

As exhibited in Listing 1, the communication through events is only possible because events carry semantic data that is used by microservices in distinct ways [72]. For instance, although the payment data is sent by the customer in a checkout request (leading to a `checkout_cart` event), such data is only used by the *Payment* microservice. This requires both *Cart* and *Order* microservices to pass along payment data in the `checkout_cart` and `place_order` events, respectively.

```
checkout_cart:{ customer_id, items:[{ product_id, qty, .. }],
    card_num, exp_date, address, discount }
replenish_stock:{ product_id, qty, timestamp, ... }
price_updated:{ product_id, old_price, new_price, ... }
product_deleted:{ product_id, type, description, ... }
```

Listing 1. Example of microservice-generated event types

It is noteworthy that messages exchanged asynchronously among microservices are usually semantically classified as commands or events by practitioners [93]. The former indicates a call for action for the receiver microservices, representing an intent from the producer microservice. For example, `checkout_cart` and `place_order` messages represent commands. The latter represents a fact that has taken place in the past, such as `price_updated` and `product_deleted`. In practice, they do not alter the behavior of microservices or systems supporting microservices such as **Message-Oriented Middleware (MOM)** or **Database Management System (DBMS)**. In this work, we use the term event to refer to both.

2.3 Current Solutions and Challenges

The state-of-the-practice in event-driven microservice architectures often employs traditional web application frameworks such as Spring, .NET, and Node.js to implement microservice components and rely on MOM such as message queues and log-processing systems to enable persistent asynchronous communication [115]. Application programmers can also design event-driven microservices through Function-as-a-Service systems [120] such as AWS Lambda [9] and actor systems such as Orleans [24]. However, these have not arisen as dominant practices in event-driven microservice architectures [12, 92, 184].

Besides the apparent attractiveness of a decoupled design, developers often end up encountering challenges in managing events independently of the technologies adopted, as we discuss below. As events are generated asynchronously, that is, producers do not wait for consumers' consumption, developers must account for the anomalies that possibly may arise in event delivery, as described by the microservice adopter Nubank [142]:

> "you have late-arriving events, you can make a payment that we receive that should be credited on Friday, we need to time travel back, re-play the events and figuring out the balance that does not invalidate the invariants we have."

Besides, it is often reported that developers necessitate making sure anomalies in the event order do not lead to problems in their microservices, as explained by an Uber team [59]:

> "If it consumes an event that is not in sequence, our processing logic identifies the version mismatch and we retry the event a number of times."

Duplicated events can also pose risks to internal microservice states, as noted by Netflix [172] below:

> "Consumers should have idempotent behavior to account for any potential infrastructure and producer mishaps [..., and] should not repeat compute heavy operations for already-processed messages."

Events may not even arrive or fail to be processed by microservices, which can lead to misleading assumptions about the global state, as noted by a Wix developer [180]:

> "[...] developers noticed that the inventory service database does not reflect the actual inventory levels of some products. Was the OrderCreated or PaymentCompleted event produced correctly?"

Turning to our example scenario, if `price_update` events are processed in different orders, that can lead to incorrect product prices. Besides, supposing that the `product_removal` is not delivered to and processed by both *Order* and *Stock*, that leads the system to an incorrect state. In addition, if `place_order` is delivered more than once, that can lead to overcharging customers. An abnormal burst of events can also pose challenges to the system if consumers cannot cope with the arrival rate of events. As new events are generated, they often incorporate some data contained in previously consumed events, leading to arbitrary types, content, and growth in the sizes of the events as they flow across different microservices, posing performance challenges to the system.

In this work, we delve into these and several other types of challenges that affect event management in microservice architectures.

3 Study Design

Although event-driven microservices are a popular architectural style in industry settings [110, 115, 154], existing studies mostly overlook the practical implications of adoption and the challenges brought about by an asynchronous and event-based design for microservices. Thus, the goal of this work is to *identify the practices adopted and challenges faced by practitioners developing software systems using event-driven microservice architectures.*

To achieve this goal, our study focuses on answering two **Research Questions (RQs)**, as follows:

— *RQ1. What is the state of the practice on managing events in microservice architectures*? *Rationale*: Despite the popularity of event-driven microservice architectures, in the literature, it is little known why practitioners adopt this architectural style. Also, the literature is scarce

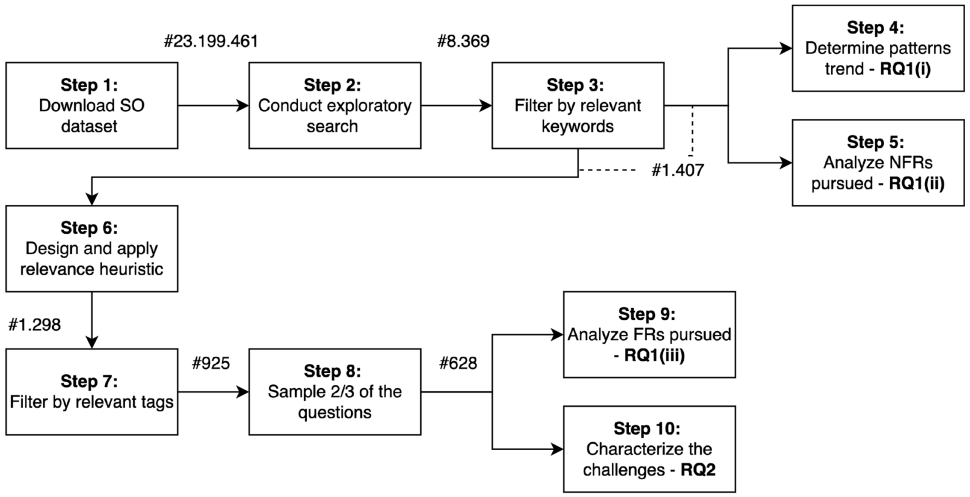


Fig. 2. An overview of the methodology.

on describing the practices adopted by them to manage events in microservice architectures, allowing practitioners to understand technological trends and popular implementation patterns associated with event management adoption. Thus, with this RQ, the insights we want to gain are threefold: (i) Collect the most popular patterns associated with event management adoption and identify what (ii) NFR and (iii) functional requirement are either enabled or facilitated by managing events in microservices.

— *RQ2. What challenges do practitioners encounter when managing events in microservice architectures*? *Rationale*: Once we understand the current practices on developing software systems using event-driven microservice architectures, we focus on identifying what particular challenges developers face when developing and operating them. In this question, we delve into the specific issues and impediments microservice developers face in practice while managing events.

To answer these two RQs, we use SO [185], a popular question-and-answer online platform where practitioners seek technical assistance on issues they face in their day-to-day activities to understand the challenges of managing events in microservice architectures. Our methodology is similar to the ones adopted in previous studies [13, 213, 216] to collect, filter, and analyze questions. However, our focus is on event management in microservice architectures. Figure 2 shows an overview of the methodology employed. The boxes indicate specific steps, and the arrows indicate both step precedence and data dependence. The number of posts resulting from the heuristics and filters applied in a step is exhibited in the subsequent edge of the same step. We detail the procedure as follows.

3.1 Data Collection

3.1.1 Downloading SO Dataset. We started by downloading the entire SO dataset S_{all} from the official Stack Exchange Data Dump [183] available when we started this study (15 September 2023). The dataset includes 23,199,461 questions dated from 31 July 2008 to 31 December 2022.[2] Every

[2]We excluded the year 2023 from our analysis as it was still a year in progress when this work was carried out.

Table 1. Relevant Tags Extracted from SO

Tag	# Questions	Tag	# Questions
microservice	895	messaging, saga	22
architecture	124	database	20
event-sourcing	90	masstransit, publish-subscribe	18
apache-kafka	78	event-handling, asynchronous	17
domain-driven-design	76	eventual-consistency, soa, distributed-transactions	15
CQRS	71	vert.x, distributed-computing	11
rabbitmq	54	distributed-system, authentication	10
event-driven	43	redis, google-cloud-pubsub	9
events	38	azure-service-fabric, event-bus, messagebroker, authorization	8
event-driven-design	36	spring-cloud-stream, system-design, websocket, azureservicebus, socket.io, web-services	7
design-patterns	31	spring-kafka, bounded-contexts, database-design, event-based-programming, aws-lambda, google-cloud-platform, transactions, logging, amqp	6
message-queue	23	apache-kafka-streams, software-design, security, identityserver4, integration, <service>	5

question in the dataset has title, body, and a "tag" metadata, which denotes the topics on which the question lies.

3.1.2 Exploring Initial Dataset. We start with a general tag set to include as many relevant questions as possible. Thus, we use T_{ini} = {"microservice"}, resulting in 8,369 questions, from which we performed an exploratory search. We observed that many questions were unrelated to event management, requiring significant effort to filter out. We also observed that relevant questions would mention the keywords "microservice" and "event" in either the title or body.

3.1.3 Filtering by Relevant Keywords. With the above insight in mind, we filtered the questions in T_{ini} that contain the keywords "microservice" and "event" in their title or body, leading to a total of 1,407 questions (denoted as S_{rel}). We extracted 566 tags from S_{rel}, which we denote as T_{key}. Table 1 shows the distribution of questions per tag in S_{rel} (only the tags with five questions or more).

The first two authors jointly examined the candidate tags and observed that many of them relate to techniques, patterns, and technologies applied to managing events in microservice architectures, such as *EDA*, *pubsub*, *message queue*, and *event sourcing*. Even tags unrelated to event management (e.g., *authentication*, *logging*, and *database*) were often accompanied by tags related to event management, indicating possible correlated issues. Both observations gave us the confidence to proceed with further analysis.

3.2 Analyzing the State of the Practice

3.2.1 Patterns Trend. Based on a popular collection of patterns for microservice architectures [167], we extracted from the questions in S_{rel} the patterns for microservices mentioned by developers. It is noteworthy that we also included in the analysis the different users' posts (i.e., responses)

for each question, totaling 3,142 entries. We took extra care in analyzing the dataset to embrace all possible forms of writing the same pattern (including typos), synonyms, and acronyms. For instance, we identified the pattern "**Command Query Responsibility Segregation" (CQRS)** via "cqrs" or "CQRS." Section 4.1 answers RQ1(i).

3.2.2 NFRs Trend. We also analyzed the most common NFRs mentioned by practitioners. We use as a reference the NFRs listed in Chung et al. [36]. We started by searching keywords related to NFR terms in S_{rel} (including posts) and incrementally introduced acronyms and synonyms. After introducing each keyword, we evaluated randomly selected related questions to validate whether the classification of the NFR fit the questions' context. We continued this process until exhausting the set of possible NFRs.

We understand that an NFR keyword search does not necessarily mean that such a requirement is part of the practitioners' problem. However, our analysis observed that such cases are often exceptions rather than rules, thus not undermining the analysis of the most pursued NFRs regarding event management in microservices. We answer RQ1(ii) in Section 4.2.

3.2.3 Functional Requirements Trend. We were also interested in understanding the specific functional requirements pursued by practitioners that are either enabled by or facilitated by managing events. Unlike NFRs, though, we noticed that uncovering functional requirements often necessitated a thorough, detailed analysis to uncover the application scenario and, consequently, the functional requirement related to event management.

Relevance Filter. Given the high number of questions, manually examining all of them would demand a substantial amount of time, which could make reporting this study's evidence in a timely fashion impractical. Therefore, at this point, we decided to apply a relevance filter. First, we use a relevance heuristic ν to filter out questions with tags that are employed to a much lesser extent compared to others. As in previous studies [13, 213], we considered only the tags whose ν are higher than 0.005, leading to a set of 1,298 questions and 93 associated tags:

$$\nu = \frac{\text{\# of questions with tag t in } S_{rel}}{\text{\# of questions in } S_{rel}}.$$

Next, we focused on prioritizing the analysis of questions tagged with event management technologies (e.g., rabbitmq, kafka, and masstransit) and techniques (e.g., CQRS, event-sourcing, and event-handling) rather than questions tagged with technologies not directly related to event management (e.g., python, java, php, elasticsearch). Although the former also presents problems in the event management domain, which is thus part of our problem scope, we observe that filtering by related event management tags led more often to questions revealing challenges within our study scope.

These two filters combined led to 53 tags (shown in Table 1) with a total of 925 questions. To proceed with the manual examination, we randomly sample 628 questions, which represent more than 2/3 of the relevant questions filtered.

Procedure. We adopt an open coding procedure [177] to analyze the sampled questions to pinpoint the specific functional requirements sought by practitioners inductively.

The first two authors, both of whom have years of software and data engineering experience, jointly participate in the manual examination of the questions. They analyze the sampled questions multiple times to familiarize themselves with them. In this process, many elements of the question were taken into consideration for inspection, including the title, body, code snippets, URLs, and the author's responses to other users' inquiries, which can ultimately clarify unclear points of the original problem statement (i.e., content in the body).

We observed that the user explicitly expressed the functional requirements being sought in most of the questions. For instance, in data replication cases, users would often mention that an event is generated based on some update (e.g., a user's credit card score), so other microservices can also apply this change to their databases (see *s3* in Section 4.3).

However, in other cases, the specific functional requirement is not made explicit. For instance, different terms, like "consistency," are used to denote replicating data as a functional requirement, like in the SO question below [214]:

> *Customer* and *Order* microservice both have customer details, though in *Order* microservice customer infos are stripped to only required fields. I understand there is a way to maintain *consistency of data across microservices using events.*

Therefore, upon clarifying the functional requirement, the authors give terms to represent the requirements of the questions, using terms used by developers themselves whenever possible. We also found cases in which a question contains more than a functional requirement. This way, we count the number of functional requirements independently of the number of questions. Section 4.3 answers RQ1(iii).

3.3 Characterizing the Challenges

We use the same randomly sampled question set used above to analyze specific issues that developers report and characterize the challenges of managing events in microservice architectures.

Again, the first two authors adopt an open coding procedure and analyze all the elements of each question, including comments from users other than the author's question, to carefully extract insights about practitioners' challenges. The detailed procedure is as follows.

First, we ensured the practitioner's requirements were properly clarified. The reasoning is that we noticed the user often reports a problem faced subsequent to or in the context of expressing the NFR and functional requirement sought. Since we already had the questions labeled with requirements through the previous methodology steps, our primary task remained in understanding the challenges from the problem description and the other question elements.

In many cases, the challenges could be identified by questions raised in the form of "how" or "what," as in other studies [213]. For example, one user asked [128]: "How can I process the event with the user credentials?"; and another user inquired [205]: "What's maybe not a good idea in trying to recover the current state of your domain model by replaying an arbitrary set of your events."

We also observed, though, in other cases, the users would not raise a question, but rather:

(a) express the willingness to achieve certain functionality [141]: "I want to create a third microservice that is responsible for joining the data of *ProductService* and *StoreService*";
(b) describe the possible cause of a problem [103]: "In my system two different sources can cause creating specific type of event. [...] due to replication lag"; and
(c) inform a desired correctness criterion not being met [161]: "The end price should be 100 for that product, but sometimes these events are processed in random order."

In a few cases, the question's body lacks detailed information about the user's application scenario. In those cases, we searched for additional comments from the author's post made as a response to questions raised by other SO users. In the absence of further details about the user's problem scenario, the authors made the best effort to characterize the reported issue, jointly discussing whenever a disagreement arose.

Second, once the problem statement was clarified, the authors started grouping similar problems into categories. The authors jointly iterated multiple times over the questions and categories.

Table 2. Relevant Patterns Extracted from SO

Pattern	# Posts	Pattern	# Posts
Messaging	1,056	Shared Database	50
Remote Procedure Invocation	624	Circuit Breaker	48
Event Sourcing	537	Transactional Outbox	43
Database per Service	424	Domain-Specific	42
API Gateway	249	Single Service per Host	39
Domain Event	225	Service-per-VM	38
Service Registry	221	CDC	38
CQRS	219	Backend for Frontend	32
Aggregate	207	Service per Team	32
Service-per-Container	192	Application Metrics	25
Access Token	174	Health Check API	25
Eventual Consistency	171	Externalized Configuration	24
EDA	151	Polling Publisher	23
Saga	141	Distributed Tracing	23
DDD	138	Self-Contained Service	23
Service Template	97	Audit Logging	20
3rd Party Registration	96	Transaction Log Tailing	20
Log Aggregation	54	Multiple Service per Host	20
Sidecar	53		

Whenever conflicts were observed in grouping the questions, a third arbitrator was introduced to discuss and reach a consensus. The third arbitrator (the last author) has more than 15 years of experience in cloud computing and data engineering. Finally, all questions come to an agreement, and the final categories (i.e., the challenges) are confirmed by all the participants. Section 5 answers RQ2.

4 State of the Practice (RQ1)

In this section, we describe the state of practice based on the discussions found in SO. Based on the analysis of the SO posts, we present some "Highlights," which are insights we conjecture from the data.

4.1 Patterns Trend

Table 2 exhibits the most recurrent patterns that appear in questions related to event management in microservices architectures.[3] In the top 10 most mentioned patterns, four are directly related to event management (*messaging*, *event sourcing*, *domain event*, *CQRS* [70]), one related to synchronous communication (RPC), two related to deployment patterns (*database per service* and *service-per-container*), and three related to typical microservice architectural patterns (*API gateway*, *service registry*, and *aggregate*).

We can observe that although event management patterns dominate the list, patterns unrelated to event management also appear substantially. For instance, remote procedure invocation, a synchronous communication technology, appears as the second most cited pattern. As events are processed asynchronously, the employment of synchronous communication mechanisms contrasts with the pursued benefits of events, which are often related to more efficient usage of computational resources and decoupling [72, 110, 114]. This trend is mainly due to the need for developers to

[3]For improved presentation, we show only those patterns appearing in 20 or more questions.

reply to users with the result of asynchronous computations, as exemplified in a developer's quote below. These challenges are further discussed in Section 5.1.

> "How is it possible to send the results stored in the kafka topic back to the requesting Web-Client_X?"

Popular microservice patterns that are not related to event management also appear in the top 10. Notably, the *API gateway* and *service registry* patterns are at odds with asynchronous communication in EDAs because they impose a synchronous-based communication (e.g., via HTTP). While communication among microservices occurs predominantly via asynchronous messages in EDAs, workflows involving multiple microservices (further details are provided in Section 4.3, FR2) may be triggered by external clients, motivating the use of synchronous-based APIs (e.g., HTTP) to support such cases. Requiring external clients (e.g., browsers) to produce events to a message broker leads to additional complexity for clients, breaks the encapsulation of the system, and has not arisen as a popular practice.

We also observed that the patterns *Database per Service* and *Service-per-Container* are often used synonymously by developers. The popularity of these two database deployment patterns suggests developers tend to avoid deployments that prescribe two or more microservices sharing the same database. For instance, the *messaging* pattern is only mentioned together with *Shared Database* on 12 occasions. These results confirm previous findings, in which adopting a shared database pattern would jeopardize the benefits of decoupling through events [115]. This mainly happens because microservices would compete for computational resources of the machine (or container) hosting the database server, leading to decreased performance effects. We further discuss this trend in Section 4.2.

To aid our analysis, we grouped the patterns into pairs and counted the number of times they are cited together in questions, as exhibited in Table 3. In general, the pairs' appearance is aligned with the individual numbers of Table 2. Event management patterns, such as *messaging*, *event sourcing*, *CQRS*, and *domain event*, appear substantially in pairs, followed by *Remote Procedure Invocation* and *Database Per Service*. Other interesting insights are also confirmed, like the popularity of an authentication mechanism together with service discovery. Furthermore, the combination of pairs sheds light on the most common set of patterns considered in questions involving event management in microservices. *Messaging*, *event sourcing*, *Database per Service*, *CQRS*, *Service Registry*, and *Third Party Registration* are substantially mentioned in conjunction in questions, suggesting a pattern adoption trend.

To further aid our analysis, we grouped correlated patterns into categories shown in Figure 3. Patterns associated with data management form the most prominent category, followed by communication patterns, application architecture, support-service, deployment, security, and observability. We observe three subgroups within data management, including patterns associated with data consistency enforcement, database deployment, and data querying. Interestingly, in data consistency, we observe not only event-related patterns, such as *domain event* [168] and *event sourcing* [68], but also *Sagas* [77] and *eventual consistency* [14]. That suggests developers also look for event management as an alternative to traditional, synchronous-based mechanisms for implementing consistency patterns, confirming the preliminary findings of the study [115]. More specifically, asynchronous events can enhance the performance of data consistency patterns by allowing for non-blocking interactions across microservices, which can lead to higher performance.

In the application architecture realm, EDA is substantially mentioned, an expected trend given the scope of the study. However, **Domain-Driven Design (DDD)** also appears to a lesser extent. We found that only 12 questions mention EDA and DDD in conjunction, suggesting these two patterns are not popularly adopted together when it comes to event management in microservices.

Table 3. Relevant Pairs of Patterns Mentioned in Event Management Extracted from SO

Pair of Patterns	# of Posts	Pair of Patterns	# of Posts
Remote Procedure Invocation and Messaging	278	Service Registry and Remote Procedure Invocation	69
Messaging and Event Sourcing	202	Messaging and CQRS	68
Remote Procedure Invocation and Event Sourcing	145	Service-per-Container and Remote Procedure Invocation	67
Messaging and Database per Service	132	Messaging and Aggregate	67
Remote Procedure Invocation and Database per Service	125	Remote Procedure Invocation and Domain Event	67
Event Sourcing and Database per Service	124	Messaging and Eventual Consistency	64
Event Sourcing and CQRS	122	Saga and Messaging	60
Service Registry and Messaging	114	Domain Event and CQRS	58
Service Registry and 3rd Party Registration	90	Messaging and Access Token	57
Event Sourcing and Domain Event	85	Domain Event and DDD	56
Messaging and API Gateway	84	Remote Procedure Invocation and EDA	53
Remote Procedure Invocation and API Gateway	81	CQRS and Aggregate	51
Event Sourcing and Aggregate	81	Event Sourcing and API Gateway	51
Messaging and Domain Event	81	Eventual Consistency and Event Sourcing	50
Database per Service and CQRS	72	Service Template and Remote Procedure Invocation	50
Service-per-Container and Messaging	72	Remote Procedure Invocation and CQRS	50
Messaging and EDA	71		

As EDA often prescribes event-triggered specific functions [72], our analysis suggests that this direct mapping between an event type and an application function (aka business logic) demotivates the modeling of extra modularity layers as prescribed by DDD.

We also observe the representativeness of patterns associated with deployment, such as *sidecar* and *service-per-container*, security with *access token*, and observability with *log aggregation*. The categories and related patterns exhibited in Figure 3 form the bulk of patterns cited by microservice developers. We explore further these phenomena in Section 5.

Finding 1. There is a specific subset of patterns that appear frequently in event management questions. These mainly relate to database resource isolation, data consistency, and asynchronous messaging. Interestingly, a significant number of questions involving event management are also accompanied by mentions of patterns not directly related to event management. For instance, Remote Procedure Invocation was mentioned in 44% of the posts, Database per Service in 30%, and API Gateway in 18%, suggesting users often attempt to blend heterogeneous patterns in their deployments.

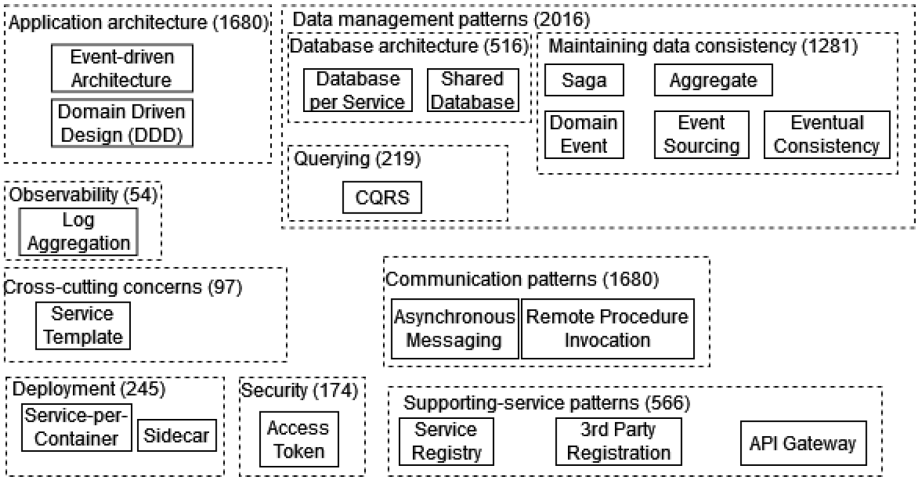


Fig. 3. Most popular patterns in event management in microservices.

Table 4. Relevant NFRs Extracted from SO

NFR	# of Posts
Consistency	312
Decoupling	303
Scalability	249
Performance	194
Modularity	170
Traceability	164
Security	154
Fault Tolerance	138

Table 5. Relevant NFR Pairs Extracted from SO

Pair of NFR	# of Questions
Performance (24.23%) and Scalability (18.87%)	47
Decoupling (15.51%) and Scalability (18.87%)	47
Consistency (14.42%) and Decoupling (14.85%)	45
Consistency (12.5%) and Scalability (15.66%)	39

4.2 NFRs

Table 4 presents an overview of the NFRs extracted from SO. To augment our discussion, we also analyze the most common pair of NFRs that appear together in questions, which is presented in Table 5. We discuss these pairs of NFRs to identify key patterns or interdependence for a better understanding of the pursued NFRs in EDA.

Consistency and Decoupling. Consistency appears to be the most cited NFR in event management questions, in sync with the microservice patterns collected in the last section. A popular concern we noticed in this regard is the unmet expectation of certain events to arrive. For instance, one developer mentions a consistency problem observed whenever the delivery of events lags behind [26]:

"When an *event isn't received* in the worker, the counters start to '*drift away*' *from the true* MySQL count. [...]"

We observe that most developers express awareness that there is a natural delay to be expected. In a few cases, though, concerns over "how up to date" a given microservice are implicitly raised, as exemplified by the same developer, as follows:

It is *expected* that there is a *consistency delay* for this type of data. How up-to-date the data is can even be figured out and included in the responses for stats data.

We further discuss the tension between the eventual delivery of events and application correctness in Section 5. Furthermore, we found that decoupling is the NFR most correlated with consistency. Analyzing the posts in which consistency and decoupling are mentioned together (45 posts), we observed that developers often identify tradeoffs in pursuing the two NFRs in conjunction. The following extracted SO thread exemplifies this trend. A developer presents their application requirements, starting with decoupling concerns [56]:

While each microservice generally will have its own data—certain *entities* are required to be *consistent across multiple services.*

I do not want shared database architecture, where a single DB manages the state across all the services. That *violates isolation and shared-nothing* principles.

And then highlights a possible mechanism to ensure cross-microservice consistency [56]:

"I do understand that, a microservice can publish an event when an entity is created, updated or deleted. All other microservices which are interested in this event can accordingly update the linked entities in their respective databases."

However, the developer realizes some inner drawbacks [56]:

"however it leads to a lot of careful and *coordinated programming effort across the services.* Can Akka or any other framework solve this use case? How?"

Another developer responds to the thread, acknowledging the tension between decoupling and consistency and the lack of principled solutions to the problem [21]:

I think there are two main forces at play here:

Decoupling—that's why you have microservices in the first place and want a shared-nothing approach to data persistence.

Consistency—if I understood correctly you're already fine with eventual consistency.

I don't know of any framework to do it out of the box, probably due to the many use-case specific tradeoffs involved.

Performance and Scalability. Consistency and decoupling NFRs appear together in 45 posts in SO (see Table 5). Despite being more common when analyzed individually, when analyzed together, they are less frequent than the pairs *performance and scalability*, and *Decoupling and Scalability*. We observe developers express concerns on varied performance topics. For instance, both developers below face a problem when the event processing rate exacerbates the processing capacity of consumer microservices. We delve into performance issues on managing events in Section 5.3.

(DEV#1) Service A listens to a rabbit queue and sends an http request to service B (which takes a couple of seconds). Both services scale based on the number of messages in the queue.

> The problem is that the *requests from A to B are not balanced.* [...] That obviously causes *low performance and timeouts.* [117]
>
> (DEV#2) My problem is the queue. *I can't get* an easily *scalable queue that guarantees ordering* of the messages. It actually guarantees "slightly out of order" with at-least once delivery [...] [33]

The same developer continues then:

> "But it turns out that using this solution, it will destroy *performance* when events are produced at high rates (I can use a visibility timeout or other stuff, the result should be the same)."

Fault Tolerance. We also observe developers consider the adoption of event management technologies as a mechanism to increase fault tolerance in their microservice architectures. In the example below [178], a developer inquires other users about whether Kafka can provide appropriate fault-tolerance support. We discuss failures and their possible impact on microservices in Section 5.1.

> "I am working in a project that starts creating *independent deployable services.* The service we are creating should be *resilient* with an *24/7 uptime.*"
>
> "In this case Kafka will be used as an *event system.* What do you think about the requirements and the usage of *Kafka to get a highly available and resilient* application?"

Modularity, Traceability, and Security. These three NFRs also appear frequently in posts. This was expected due to their high importance to any type of software system. Modularity is closely related to decoupling, which is discussed above. Traceability is paramount due to the substantial data and message exchanges through events across microservices, often deployed in disparate nodes. This data processing distribution exacerbates the importance of tracing, as exemplified by a developer [87] as follows.

> "Assume four services: s1, s2, s3, and s4. So a start event goes like s1 to s2 to s3 to s4. *How to know the status* (in terms of percentage) like s2 is done for a message then consider it 50% completed?"

In a similar manner, a loosely coupled architecture like EDA can introduce challenges in ensuring security criteria are enforced seamlessly across disparate microservices. For example, user authentication and authorization mechanisms, as well as techniques for data privacy, may vary across microservices, preventing a holistic enforcement of security properties.

> *Finding 2.* Consistency, decoupling, and performance appear as the most mentioned NFRs. However, developers suggest there are tradeoffs on properly meeting them in event-driven microservice architectures.

4.3 Functional Requirements

In this section, we discuss the most recurrent functional requirements sought by microservice developers on which event management is applied (denoted by *FR*[*0–N*]). Table 6 summarizes the number of functional requirements per question.

FR1. Propagation of State Updates. We observe that the majority of questions contain a requirement related to propagating state updates in the form of events. A developer summarizes this practice as follows [56]:

Table 6. Relevant Functional Requirements Extracted from SO

Functional Requirement	# of Questions
FR1. Propagation of State Updates	78
FR2. Multi-Microservice Workflows	66
FR3. Data Integrity Maintenance	33
FR4. Replaying of Events	22
FR5. Query Processing	20
FR6. Data Replication	17
FR7. Cache Management	12
FR8. Task Scheduling	5

> "A microservice can publish an event when an entity is *created*, *updated*, *or deleted*. All other microservices which are interested in this event can accordingly *update the linked entities* in their respective databases."

It is worth noting that in this question, we only take into account the questions where the developers' intention of propagating the events is either not expressed explicitly or not clear from the discussions. However, it is natural to deduce that these state updates, represented as events, are used by other microservices to achieve other functional requirements, as we discuss in the next FRs.

We also observe the prominence of event sourcing, domain event, CQRS, and aggregate patterns in questions related to this FR. Although targeting different objectives, these patterns are based on the propagation, by producer microservices, and the subscription, from consumer microservices, of state updates in the form of events.

These patterns often involve modeling intermediate states as part of the event payload. However, we cannot guarantee that intermediate states are represented in the event payload in all posts where such patterns are mentioned, either due to insufficient context or technical details provided.

FR2. Multi-Microservice Workflows. We also observe a substantial number of developers reporting use cases where a business transaction requires a composition of microservices via events. For instance, question 42,140,285 exhibits that an event generated by a microservice triggers an operation in another microservice [39]:

> "*Order [microservice]* receives an order request. It has to *store* the new Order ([record]) *in its database* and *publish a message* so that *Payment service* realizes it has to charge for the item."

Different from the previous scenario, we observe that developers in this case have an expectation about the completion of the triggered computations. In this functional requirement, the generation of an event acts as a command, a call to action, for downstream microservices, manifesting the need to perform operations, often involving the operations in their private states.

In relation to patterns, Sagas [77], messaging, and event sourcing [68] appear more frequently. The popularity of Sagas in this FR is not a surprise, since in the presence of failures or transaction aborts, preceding microservices may require withdrawing some operations, that is, compensating previous actions. Typical NFRs associated with multi-microservice workflows that were observed relate to consistency, decoupling, and scalability.

FR3. Data Integrity Maintenance. We observed questions where developers highlight how an event is used to maintain the integrity of a microservice's state. An example is provided as follows [199].

"I have a web-api-endpoint that receives orders that an *OrderMS* is responsible to handle. When order is put[,] Inventory must be updated so *OrderMS* publish an event to subscribers [...] and *InventoryMS will update the inventory* due to it is subscribing to current event/message."

Another developer explains the intention to achieve functional dependency across microservices by propagating events [214]:

"Customer and Order microservice both have customer details, though in Order microservice customer infos are striped to only required fields. I understand there is a way to maintain consistency of data across microservices using events."

In line with FR1, by subscribing to state updates in the form of events, microservices can maintain data invariants in their state. Again, Sagas appear as an important pattern to safeguard data invariants in the presence of failures or aborts in remote microservices. Consistency is the primary NFR related to the FR.

FR4. Replaying of Events. We observe that developers seek to replay (i.e., resending) past application-generated events in two cases:

(i) Synchronizing states. Microservice applications evolve over time. As new microservices are incorporated into the topology, it may be the case that these are required to synchronize with the current application state, as described by a developer [206]:

"When I add *new service* to a set of already running services, I need upstream dependencies to send all the messages from the past to the new service so it could *align its state* with the one of the whole system."

(ii) Troubleshooting and Auditing. As the communication abstraction in event-driven microservices, it is natural that the re-execution of the event flow may assist in further understanding the application behavior, as explained by the following developers:

(DEV#1) I want to *replay events* on an invoice where the I want *to see all actions done* by a specific employee on the balance. [152]

(DEV#2) The intent for this pattern [(event sourcing)] is to provide an *audit trail of all events* that took place while the patient was in the hospital. [189]

There is a correlation with this and FR1 as many patterns appear in both with a similar degree. This is expected because their root objectives are shared, propagating events. However, in this FR, developers expect that events are not lost, thus stored reliably in order to be propagated again. Key NFRs observed in questions associated with this FR are traceability and security.

FR5. Query Processing. Developers tend to report the implementation of data processing tasks at the application layer. In this case, they implement queries that operate over the payload of subscribed events, as exemplified by the following SO question [141]:

"I want to create a third microservice that is responsible to *join the data of ProductService and StoreService* in order to retrieve all the available products. Here, CQRS pattern seems like the best solution: I will create a materialized view and I will synchronize it using domain events published by the other two microservices."

Typically, distinct event streams are aggregated to form queries or views. As suggested by the snippet above, CQRS [70] is predominantly found associated with this FR. Other relevant patterns include event sourcing, domain event, aggregate, database per service, and messaging. This event stream processing pattern benefits decoupling since it prevents microservices from querying microservice states directly and breaking the desired state encapsulation allowed by events.

FR6. Data Replication. We observe that the propagation of state updates is also an enabler of data replication. By subscribing to state updates in the form of events, consumers can maintain their own view of the state managed by other microservices without requiring them to pull updates, thus avoiding the overhead entailed by synchronous calls (e.g., RPCs) [202]. For instance, in the following question [109], the developer explains that:

> "Service A could raise an event whenever a new student is registering. Service B will consume the event and *stores student info in its* [*own*] *db*."

Our analysis reveals that not all cases of propagation of state updates will definitely lead to data replication, but all cases of data replication via events necessarily require propagation of state updates. The reason why this FR is intrinsically related to patterns found in FR1, like event sourcing, domain event, aggregate, and messaging. In addition, database per service appears frequently, indicating the association of this functional requirement with decoupling, modularity, and fault tolerance.

FR7. Cache Management. In line with FR6, another reported scenario lies in maintaining caches based on subscribed events, as explained by a developer as follows [196].

> "When C starts up it needs to *load all of the current data from P into its cache*, and then subscribe to change notifications. (In other words, we want to synchronize data between the services.)"

FR8. Task Scheduling. We encountered cases where events are used as abstractions to trigger remote tasks and manage the lifecycle of long-running jobs. As example scenarios, we highlight the following:

> "That job info is placed in a RabbitMQ message and sent off by the RabbitMQ Producer [..] A RabbitMQ Consumer receives message with the job info and calls the class that is responsible for Executing the *long-running job*, the job status is updated to IN-PROGRESS" [148].

> "The idea is to have the REST API immediately post a message to a queue, with a background worker role picking up the message from the queue and *spinning up multiple backend tasks*, also using queues. REST API [..] generates a GUID and attaches that as an attribute on the message being added to the queue" [198].

This FR is linked to NFRs like load balancing, performance, and fault tolerance, and a recurringly mentioned pattern is eventual consistency. The NFRs indicate the ability of events to represent independent processing units that can be balanced evenly across microservices, benefiting performance. Regarding fault tolerance, a failed microservice does not affect other processing tasks; therefore, the tasks are eventually completed.

Finding 3. Event management is applied to fulfill varied functional requirements, such as communicating data updates, composing microservices functionalities, and processing queries, indicating the heterogeneous applicability of events.

5 Event Management Challenges (RQ2)

Having discussed the state of practice of event management, we focus on the challenges developers face when managing events in their microservice deployments. Figure 4 summarizes the challenges we extracted from SO. The leaf nodes represent the specific challenges, whereas their parent nodes represent the categories to which a challenge belongs. For example, the *Performance* (Section 5.3) category comprises three specific challenges: event processing overhead (CP1), large event payload

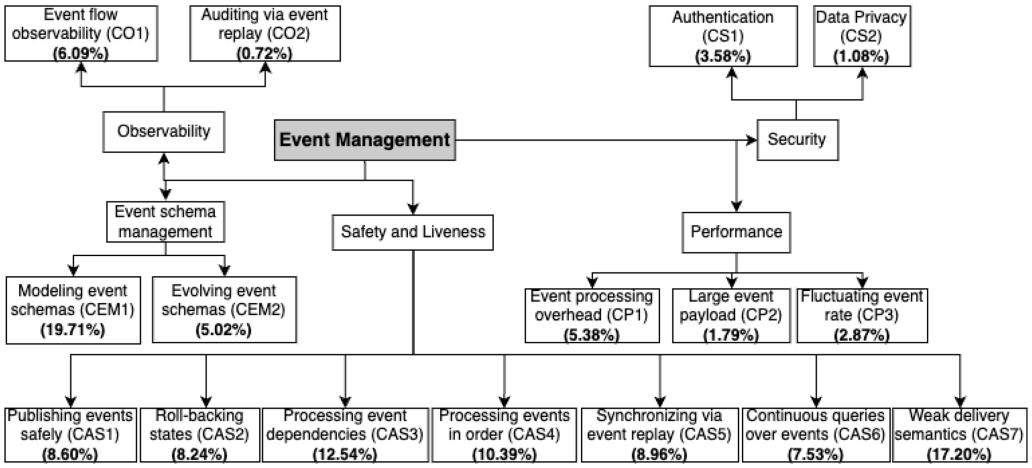


Fig. 4. Overview of challenges in managing events in microservice architectures.

(CP2), and fluctuating event rate (CP3). In total, our analysis led to 5 categories and 16 specific challenges, indicating the heterogeneity of the problems microservice developers encounter when managing events. We discuss and exemplify, with representative developers' quotes and snippets, each specific challenge by its categories.

For every challenge, we discuss the best practices, methods, and technologies, including whether and how these can possibly help developers dealing with the challenges. Furthermore, for every problem category, we also discuss the implications of the findings related to different stakeholders. For example, in Safety and Liveness, the last Section 5.1.8 maps the findings from Sections 5.1.1–5.1.7 (i.e., the challenges) to practical and research implications to practitioners, researchers, tool builders, cloud, database, and messaging technology providers. This pattern is followed in every problem category in this section.

It is worth noting that the example quotes and snippets used throughout the manuscript were selected based on the representativeness and generalizability they reflect as part of their problem category. Besides, for most of the challenges, different instances of the problem category were discussed separately using different developers' quotes to allow for fine-grained discussions.

The discussions focus on the analysis of the developers' intentions, root causes of problems, and their implications to safety, liveness, event schema management, observability, performance, and security, independently of the particular solution developers seem to be following. We believe this strategy is more fundamental for fomenting further research and support to developers, as can be assessed along with the implications to different stakeholders. It is worth noting that we refer to messaging technology as any system that enables microservices to exchange events through publishing and subscription mechanisms.

5.1 Safety and Liveness

Safety and liveness are properties inherent to distributed systems [116]. While safety properties prescribe that nothing bad happens, liveness properties prescribe that something good eventually happens. For instance, a traditional example of a safety property is found in the **2-Phase Commit (2PC)** protocol [132]. In 2PC, the effects of the operations of a transaction that cuts across nodes are only made available to subsequent transactions if all participating nodes agree. A common

liveness guarantee example is eventual consistency [154]. In this consistency model, nodes eventually converge to the same outcomes (e.g., data item versions). When this is achieved, it is often unbounded, making it a weak consistency model.

In this section, we use these two properties to discuss related problems in the context of event management in microservice architectures. We reveal microservice practitioners' expectations when managing events, particularly when publishing, consuming, processing, reverting the effects of, and replaying events.

5.1.1 *Publishing Events Safely (CAS1).*

Safety. The functional requirements investigated in Section 4.2 highlight that generating events is the cornerstone of event-based microservice architectures. Events are often raised in response to local operations, typifying a causal relationship. Therefore, failures in local operations must automatically withdraw the existence of a resulting event. However, we find that the event's publishing semantics are not always crystal clear for microservice developers. We highlight as follows the most common types of issues using developers' own quotes:

(i) Developers seek to acknowledge whether the event published has reached out to the message broker, for instance [39]:

> (DEV#1) "This operation [(publishing the event)] is asynchronous and *ALWAYS returns true*, no matter if the broker is down. How can I know that the message has reached the broker?"

(ii) Developers wonder whether publishing an event as part of a transaction is possible, for instance [150]:

> (DEV#2) "[...] does actually spring-kafka support JTA transactions and would it be enough to wrap RDBMS and Kafka Producer into `@Transactional` *[(ORM annotation)] methods*?"

(iii) In other cases, developers wonder how to ensure atomicity semantics when publishing more than one event as part of an operation, like the user below [124]:

> (DEV#3) "Let's say a command needs to append an event to both, the public and private user stream. *How can you make sure that both events have been appended?* Does the event store publish both, `SomeUserEventHapppendPrivate` and `SomeUsererEventHappendPublic`, to the event bus?"

(iv) Developers look for alternative ways to publish events out of the critical path of the application, such as asynchronously detecting database updates and publishing these as events, like the case below [169]:

> (DEV#4) "Should I have some kind of *background timed process* which will scan events table and publish events to SQS? Can this be processed within the WebApi application (preferable), or *should this be a separate a process*?"

Challenges related to ensuring events are correctly published appear in 13.79% of `Safety&Liveness` challenges and reflect 8.60% of the total, indicating the representativeness of the problem.

Cause. Coordinating writes to internal microservice states and messaging technologies is a synchronization problem, necessarily taking part at the application level in microservice systems. We observe practitioners seem to be unfamiliar with the guarantees provided by messaging technologies. As a result, they may fail to foresee the possible issues associated with ensuring consistency between two systems, in their case, the (producer) microservice and the message broker.

Discussion. While developers can use message acknowledgments, often provided by MOM such as RabbitMQ [204] and Apache Kafka [108], to determine whether the publishing operation has

successfully completed, we conjecture that the abstractions provided by programming frameworks may make it blurry to handle message offsets. Notwithstanding, most questions relate to ensuring safety properties related to the operations performed by the application program (often triggered by an incoming event) and the subsequent publishing of events associated with those, as we discuss next.

The BASE approach [154], prescribes how application developers can implement distributed components through message brokers in order to support partial failures and idempotence. However, the technique requires careful implementation from both the producer and consumer sides. Application safety issues can arise related to the commutativity of the operations, the data growth of metadata associated with progress tracking, the need for transaction resubmission in case of failures, and the generation of transaction unique identifiers. These challenges may explain why practitioners tend to overlook the approach.

Recent **Change-Data-Capture (CDC)** tools like Debezium[4] soften this challenge by dedicating a system to capture updates in a microservice private state (e.g., relational tables), offloading developers from explicitly publishing events. Each update is then eventually delivered to consumer microservices as an event. However, apart from the need to manage an additional system, when or whether this delivery is performed is unknown, which can lead to an additional challenge as we see in CAS7 (Section 5.1.7).

Furthermore, some database systems, like Oracle[5] and PostgreSQL,[6] offer mechanisms to publish events atomically as part of transactions. However, the benefits of state encapsulation would be jeopardized, given that all consumer microservices would depend on specific producer-managed databases, thus leaking the private microservice state abstraction.

5.1.2 Roll-Backing States (CAS2).

Safety. Once an application-generated event is published (i.e., reaches out to the message broker successfully), the event becomes available for consumption by other microservices, which in turn can consume and start processing the event. However, events may be consumed and processed by multiple microservices independently, which makes it challenging to track down the possible errors across multiple microservices and react accordingly. As a result, we observe that developers express uncertainties on how to appropriately deal with failures, such as the following cases:

> (DEV#1) "[...] the state modification is actually a complex operation across multiple microservices using the saga pattern, which needs to be rolled back if something fails. [...] *How to cleanup the state if the service modified it, but failed in the end*, e.g., due to system shutdown?" [164]

> (DEV#2) "When my Transaction is committed, normally I will dispatch `IntegrationEvent` (e.g., to the queue), but there is possibility that this *queue is down* as well, so previously just-committed *transaction has to be 'reverted'*. How?" [156]

In this sense, many answers received for related questions suggest the use of compensating actions triggered via additional events; however, some developers acknowledge that these inherit the same properties of the originating problem, as summarized by the following quote:

> (DEV#3) "If 3.a ([an operation triggered via event]) fails[,] a *compensation action* is performed; but *what about if it fails*?" [104].

[4] https://debezium.io.
[5] https://www.oracle.com.
[6] https://www.postgresql.org.

We observe that this challenge is significant in the context of `Safety&Liveness` challenges (13.22%), spanning across 8.24% of the total questions.

Cause. Synchronizing writes to multiple uncoordinated microservice private states via event-triggered actions is a largely unsolved problem. When the event is published, the event producer loses control over downstream processing due to the independent failure of microservices. Similarly, consumers are unaware of which microservice produced the event stream to which they are subscribed. In this case, we found some developers design specific event streams to communicate failures in the workflow, as we discuss in Section 5.1.2.

Discussion. As discussed in Section 4.1, the use of Sagas [77] arose as a popular mechanism to handle workflows in distributed applications like microservices. It posits the use of compensation operations to handle failures in multi-microservice workflows. Despite being considered a doable practice, publishing events that semantically represent the occurrence of a failure (for subsequent retraction of the corresponding action, i.e., a compensation) can also fail. That can lead to "dirty" intermediate states (i.e., the effects of canceled operations) remaining exposed indefinitely. On the other hand, even though upstream microservices acknowledge a downstream failure, once the local operations are committed as part of a transaction in the database, it is often not possible to rollback the transaction. Thus, upon a failure, it may be unclear for developers how to properly revert those changes.

Pushed by workflow execution engines [192], a recent industrial trend posits that microservices must be designed through explicitly specified workflow steps in order to ensure reliable execution, that is, guaranteeing that either all or no steps reflect their actions in the system. In practice, these workflow systems apply retry actions following a defined strategy (maximum retry times or timeout) and upon failing to complete a step, application-specified actions are triggered to revert the changes of previous steps. Despite not preventing the exposure of intermediate "dirty" states, the requirement of specifying the interaction of microservices from the startup contradicts the dynamicity and loose-coupling requirements prescribed by EDAs.

5.1.3 Processing Event Dependencies (CAS3).

Liveness. Events often carry a schema, or an event type, and are published in a stream (aka topic or queue) on which all events published must adhere to such a schema. We observe an interesting trend in which practitioners attempt to match distinct but correlated events at event processing time. The trend is exemplified by a developer [76]:

> (DEV#1) It is very straightforward to implement a [(event)] listener when *an action depends on one single event* ([stream]). [...] The problem arises when *OrderService has to wait for more than one event* [(from different streams)].

The developer contextualizes the case and expresses the problem [76]:

> (DEV#1) I am pooling both queues `CREDIT_AVAILABLE_QUEUE` and `INVENTORY_AVAILABLE_QUEUE`, and *both events has to be present* so I can finish an order. How can I coordinate so that *OrderService* sees both events as only one?

However, these dependencies across distinct events may require ordering guarantees from the messaging technology, which may not be present, as noted by another developer [33]:

> (DEV#2) If `UserCreated` *comes after*
>
> `ProductAddedToCart` the normal flow requires to *throw an exception* because the *user doesn't exist yet.* [...] So, the ordering problems are:
>
> No order guaranteed across events from the same event stream.

No order guaranteed across events from the same ES [(event store)].

No order guaranteed across events from different ES (different services).

These ordering problems can ultimately lead to concurrency bugs in the application code, as noted by the former developer [76]:

(DEV#1) There is a minimal chance of receiving both events at the same time[,] generating *race conditions*.

We observe that this challenge appears substantially, spanning across 20.11% of `Safety&Liveness` challenges. Besides, it spans 12.54% of the total questions, highlighting the practical significance of this event processing pattern.

Cause. Developers express that some events are semantically related and thus cannot be processed independently. That leads developers to encode synchronization primitives intertwined with application code. However, developers fall prey to weak ordering semantics across event streams. Another challenging factor is that the delivery of events can experience arbitrary delays due to network partitions, overload of computational resources, network jitter, and even the failure of producer microservices. These aspects together pose a challenge in enforcing event dependencies across distinct event streams.

Discussion. In connection with our previous discussion (CAS2), this event processing characteristic may find resemblance with the so-called workflow patterns, originally posited for business processes and web service compositions a few decades ago [99]. Recent microservice frameworks, like Dapr [52], have included in their API support for some workflow patterns, such as fan-out/fan-in [51]. However, workflow patterns often require either a domain-specific language or a common framework where all microservices are implemented, as well as the explicit specification of workflows, including event producers and consumers. These contrast with the polyglot nature of microservices and the decoupling through events sought by microservice developers through MOM.

5.1.4 *Processing Events in Order (CAS4).*

Safety. While analyzing the questions, we noticed that practitioners often expect events to be processed for every consumer. However, in many cases, this expectation is not met. Unlike the previous challenge, this challenge affects single event streams (i.e., all conforming to the same event type) rather than distinct events. A developer exemplifies the problem as follows.

(DEV#1) Suppose a product is ordered and it is *id* 80, and a series of sequential update events fired from product service to order services for that particular product [...] *The end price should be 100 for that product, but sometimes these events are processed in random order* [161].

The developer expects that the product price observed by the *Order* microservice must eventually be in sync with the product's state in the producer (i.e., the *Product* microservice). Therefore, any processing order not leading to a price of 100 as the final state is deemed incorrect. In some cases, practitioners acknowledge that an in-order processing guarantee cannot be enforced in consumers due to the uncertain semantics of producers:

(DEV#2) Since each microservice has its own event table and asynchronous worker, *we cannot guarantee that events will be sent in the sequence* in which the corresponding state changes occurred in their respective microservices [144].

In another case [106], a developer posits the use of application-generated timestamps to control the order of concurrents events:

(DEV#3) [...] When we generate these events from *different services running in different machines*, how to manage the time (timestamp) *going out of sync* across these thereby resulting in an *event order mismatch*.

These challenges appear in 16.67% of `Safety&Liveness` and account for 10.39% of the total challenges observed, highlighting the difficulty of ensuring events are processed in the expected order.

Cause. Although the majority of messaging systems guarantee the delivery of events from the same stream in order, such as Kafka through the concept of a topic partition [18], that does not exclude producers and consumers from still producing and processing, respectively, the events accounting for the expected order semantics. However, given the uncoordinated nature of microservices, these semantics are not always clear or trivial to achieve for developers operating their microservice processes independently.

Discussion. For producers consisting of concurrent threads writing to the same partition, even though Kafka serializes the messages, the correct order that the consumer must process the messages is not well-defined. Similarly, suppose multiple consumer threads pull messages concurrently from Kafka. In that case, if the order of messages across different pulls matters, it is complex and error-prone for general developers to reassemble the original stream order at the consumer side.

In most questions analyzed, we could not identify whether the message broker delivers the messages out of order or the microservice processes the event arbitrarily (not accounting for the delivery order). As the quote below exemplifies [149], event processing semantics appear unclear to some microservice developers.

(DEV#4) In [(omitted technology,)] *messages are best-effort ordering, still no idea of what they mean.* [...] Does it means that[,] giving n copies of a message[,] the first copy is delivered in order[,] while the others are delivered unordered compared to the other messages' copies? Or "more than one" could be "all"?

It is a good practice to map threads to particular queues or topic partitions for consumption [41]. However, this mapping is often application-specific, requiring fine-tuning the levels of concurrency and parallelism expected according to the data access patterns of the application, which in many cases render additional challenges for application developers [217].

Lastly, despite recent works proposing the use of timestamps [89] or unique identifiers [207] to support the identification of ordering relations across concurrently generated events, their usage by application functions is often considered complex and error-prone [116].

5.1.5 *Synchronizing States via Event Replay (CAS5).*

Safety. As described in Section 4.2, the need for replaying events often arises due to new microservices being introduced in the system. In this context, a developer explains an interesting challenge [155]:

(DEV#1) Service "A" creates event message to inform other services about changes [...] *Newly introduced Service "D" also needs to replicate data* coming from service "A." Service "D" needs all historic data[.]

The developer then continues expressing the impedance found.

([...]) other services were already running for a while, and *service "A" only broadcasts new changes*. What would be the correct solution to populate newly added service with historic data?

Furthermore, as microservices supposedly fail independently, it may be necessary to replay events from the point in time the subscriber microservice(s) failed, as described by another developer [4]:

> (DEV#2) I'm wondering what should happen if one of these events can not be delivered due to an error [...] On republishing events, should all messages be republished to all topics[,] or would *it be possible to only republish a subset*?

Challenges involving replaying events also appear significantly in `Safety&Liveness` challenges (14.37%), accounting for 8.96% of the total challenges, suggesting mechanisms for safely replaying events are strongly sought by developers but missing in practice.

Cause. Similarly to CAS2, synchronizing distributed, largely uncoordinated states via asynchronous events lies in the space of hard distributed system problems for application developers. Along the analysis, we find that the need for replaying events arises not only from replicating data and software evolution, including adding new microservices and migrating to a microservice architecture, but also when recovering from failures and fixing the outcome of bugs.

Discussion. Developers express uncertainties about how to proceed properly in the cases above, and end up implementing several ad-hoc mechanisms at the application layer to fulfill the many aforementioned application needs. As a result, these mechanisms often result in rework and create additional issues in the application.

To the best of our knowledge, although there is no widespread notion of best practice in this problem domain, application developers can benefit from best practices related to tracking event consumption progress posited by event log systems such as Kafka [111]. The mechanism is referred to as offset management [41] and prescribes the basis for arbitrary event log consumption, that is, the ability to read not only the last appended events, but any particular segments of the event log from any point in time. Developers must guarantee that sections of log segments are read accordingly to the required data and, in case of errors, guarantee that no duplicate events are read. Despite the complexity, care must be taken when accessing offsets that are no longer valid. From time to time, old logs are pruned in message broker servers to meet storage and performance constraints.

5.1.6 Continuous Queries over Events (CAS6).

Liveness. We observe that practitioners commonly benefit from the application data that is often contained in events to build a view of the application state without the need to explicitly query private encapsulated data. Such views are continuously updated based on the arriving events. However, developers express concerns on ensuring correctness, as described by a developer [141]:

> (DEV#1) When I update a *Product* entity or *Store* entity there is no problem to sync the view because I already have data on it. But what about when I receive a `ProductCreated` event?
>
> This event has only information about product and nothing else, so `stock`, `store_name`, and `store_address` *will be NULL*.

Furthermore, the nondeterminism of asynchronous events (i.e., the arrival of events may experience large delays or even never arrive) poses challenges in ensuring correct query processing results.

> How can I save this event in my view? *Should I save incomplete data somewhere else* and update my view *when I will receive complete data*?

We also observe that some developers even implement code structures that resemble query operators (e.g., filter, join, and aggregations) [218]. This is particularly problematic since, as software

evolves over time, changing requirements can also undermine providing correct query results based on events, as exposed by another developer [97]:

> (DEV#2) I have *new requirement*—calculating maximum all time temperature per sensor.
>
> I have prepared *new microservice* that creates `KTable` ([Kafka table abstraction]) aggregating temperature (with max) grouped per sensor.
>
> Simply deploying this microservice would be enough if input topic had infinite retention, *but now maximum would be not all-time*, as is our requirement. [...] How to design the solution? (67074772)

The impediment found above relates to the inner characteristic of stream processing engines, which rely on windows and notions of time to avoid querying all the historical data in order to provide fast responses [91]. Continuous queries over events appear frequently in Safety&Liveness challenges (12.07%) and correspond to 7.53% of the total questions, highlighting the significance of this challenge.

Cause. Developers build customized solutions to create real-time views based on observed events, often through patterns like CQRS, often not realizing that providing correct query results with this pattern incurs hidden challenges. Besides implementing the query operators and window semantics, developers must ensure that duplicate events (further discussed in Section 5.1.7), crashes, and delays do not impact the query results.

Discussion. For over two decades, continuous queries and stream processing systems have been a mainstream research topic in the data management community [73], resulting in industry-strength solutions like Materialize [127], Flink [66], and Kafka Streams [108]. These systems allow users to declare a query that is continuously updated based on incoming streams. However, we notice that microservice developers often do not mention stream processing systems and prefer patterns like CQRS.

In terms of best practice, stream processing systems are designed to handle the gamut of problems that arise in maintaining continuous queries. However, apart from mentions to Kafka Streams [108], we did not observe substantial adoption of stream processing engines. We conjecture that microservice developers are either unaware of or may find it difficult to map their application logic to stream processing engines. Another adoption barrier can be addressed to unfamiliar stream processing concepts, such as windows and punctuations [91].

Libraries to facilitate the implementation of the CQRS pattern have arisen in the meantime. Drasi framework [182], for example, extends Dapr to provide automatic update of pre-declared queries. We believe such tools can bridge the gap of incorrect CQRS implementations at the cost of eventual consistent updates, which is already assumed by the CQRS pattern.

5.1.7 Weak Delivery Semantics (CAS7).

Liveness. Messaging technologies often guarantee at-least-once delivery by default, leaving the responsibility of achieving at-most-once or exactly once delivery semantics to application developers [73]. However, by resorting to at-least-once delivery semantics, developers can face unpredictable challenges, as exemplified by the developer as follows [105]:

> (DEV#1) I was able to collect the data [(published as events)] from another microservice, but I noticed that if I terminate the [(consumer)] process and run it again, I get the data back, PLUS another copy of the same data right under it. [...] *I got the payload sent to me twice, and I only want to see it once.*

As the developer's quote shows, a problematic aspect of at-least-once delivery is that events can be delivered more than once. That forces consumers to either make their microservices idempotent

or fall prey to the undesired effects of duplicate event processing. The following microservice developer expresses another interesting case [40]:

> (DEV#2) The only way I have *to know* that the *payment has been processed* by *Payment* service *is by expecting an answer event* (`Payment ok|failure`).

However, in the absence of an event that confirms that a payment has been processed, the developer comes up with its own customized solution:

> If it hasn't gotten an answer in some time, [that] *forces me to implement a retry mechanism* in the *Order* server, [that is,] retry with a new `Payment` event.

The developer ended up dealing with an unexpected outcome:

> [However,] this also *forces me to take care of duplicated messages* in *Payment* service in case they were actually processed but the answer didn't get to the *Order* service.

Another popular issue with resorting to events for asynchronous processing lies in the fact that computations are often initiated based on client requests. These online requests require a timely response to users and challenge developers to match the eventual arrival of asynchronous events (i.e., the response of the client-triggered asynchronous computation) with the corresponding waiting client. A developer describes the problem below [7]:

> After publishing the event[,] my booking *service can't block the call and goes back to the client* (front end).
>
> How does my client app will have to check the status of transaction? Does it poll every couple of seconds? Since this is distributed transaction and *any service can go down and won't be able to acknowledge back*.
>
> In that case how do my client (front end) would know since *it will keep on waiting*.

Dealing with weak delivery semantics is the most popular challenge in `Safety&Liveness` category (27.59%). By appearing in 17.20% of the total questions, this evidences the difficulty of managing events under weak delivery semantics.

Cause. Event losses and delays can be observed due to many factors. In addition to arbitrary failures (e.g., consumers crashing), during a rebalance in Kafka brokers, consumers can experience delays in event delivery [64]. Besides, the network can become unstable, leading to delays in network package deliveries. Therefore, a delayed event may have a gamut of causes that are not necessarily related to crashes in the consumer microservice itself.

Discussion. In an attempt to ensure the completeness of operations across microservices, developers often rely on custom-made solutions, such as the retry mechanism shown above. However, ad-hoc retries can lead to the additional burden of dealing with duplicate events, which forces developers to implement additional mechanisms at the application layer to impede duplicate events from being processed, only exacerbating the problem.

Microservice frameworks like Dapr offer transparent event consumption tracking for applications, using automatic retries to achieve at least once processing. However, it cannot ensure exactly once processing, which may indefinitely leave non-idempotent applications in an inconsistent state. As a result, developers are still responsible for ensuring the consistency of application state, and no duplicate events are produced/processed in the event of failures. Similar guarantees are found in other industry-strength frameworks like Durable Task Framework [47].

Finding 4. Developers encounter a myriad of challenges on ensuring events are processed correctly. These span the entire lifecycle of an event in a microservice architecture, including queuing, delivering, storing, processing, and synchronizing events.

5.1.8 Implications. Given the wide range of safety and liveness challenges, we discuss the implications in two groups. The first covers event publishing and delivery semantics, and the second relates to the challenges of synchronizing microservices' events.

CAS 1, 2, 4, and 7. Developers lack a comprehensive specification to guide them with the variety of event processing semantics and in dealing with failures in asynchronous, distributed microservices, suggesting they are in need of supporting tools. *For messaging technology providers and framework developers*, their documentation can emphasize the publishing and delivery semantics offered more clearly, including, but not limited to, the possible failure scenario microservice developers must consider and example workarounds to mitigate some of their effects in the microservice state. In addition, these should be provided in a language suitable for developers who are not familiar with distributed and asynchronous systems.

For database providers, explicit APIs for rolling back microservices' underlying databases to a state prior to processing a given event can help alleviate the burden of dealing with online failures in event-based workflows. *Researchers* can systematically characterize the guarantees provided by popular messaging technologies and enhance code analysis tools to better alert developers about potential hidden shortcomings of the different guarantees.

CAS 3, 5, and 6. The need to process and match distinct event streams generated by different producer microservices spans different functional requirements, such as replicating data, processing distinct but correlated events, and maintaining queries over event streams. Together, they form 29.03% of the total challenges, highlighting the practical significance of these event processing patterns. To fulfill these, developers often hold the assumption that events will arrive in a timely and exactly once manner. However, events may either take an arbitrary time to arrive or never arrive. Besides, even if events arrive, they may be repeated. The mismatch between expectations and reality leads to frustrations in implementing the requirements above. Thus, microservice developers must be vigilant with designs that favor the existence of such event processing patterns. Rethinking service boundaries [1] may lead to some of the events involved in intricate event processing patterns being merged or eliminated.

Providers of messaging technologies and cloud computing services, and framework developers can offer further guidance at the documentation and API levels to alert developers regarding the potential dangers of some event processing patterns. Potentially, message technologies can devise better APIs to support developers making sure event dependencies are met. For example, an API to wait for multiple dependent events instead of individual events.

Researchers can improve static and dynamic analysis tools to detect certain code properties and execution traces that may lead to the harmful scenarios discussed. Another direction could be developing automatic tools to enforce safety properties on event processing. For example, Lesniak et al. [118] extend the message queuing system with a programmable interface where developers can specify event processing order, which will be enforced automatically.

All CAS. Researchers can devise or extend existing programming models to account for use cases that necessitate correlation between distinct event streams and abstract publishing and delivery semantics from the microservice code. For instance, they can take inspiration from the dataflow model [3], commonly used in stream processing engines, to accommodate the missing dynamic topology, non-blocking, isolated failure model found in real-world event-driven microservices. That can simplify the programmability of microservices and favor the adoption of tricky event stream

processing patterns. Additionally, this programming model can unify the design and deployment of event-based microservices across cloud providers and execution platforms to facilitate the migration and hybrid execution of microservice applications across message technology providers, an emerging trend in cloud computing [37].

5.2 Event Schema Management

Microservice practitioners model events for external interaction. In other words, the decoupling nature of a microservice design requires producers to encode all the necessary data so that consumers do not need to query the producer for additional data when processing such an event. Furthermore, the events generated by microservices must comply with a data type to be correctly processed by consumers, and, as with any other data type, it can evolve over time. This section discusses the challenges associated with modeling and evolving event schemas in microservice architectures.

5.2.1 Modeling Event Schemas (CEM1). We observe that how events are modeled and what they represent semantically may dictate the design and performance of microservices. For example, a developer looks for recommendations for event design explaining the following scenario [35]:

> Say you have a micro-service architecture where multiple services produce and consume *unit_statuses. There are multiple ways to design this.*
>
> The person continues explaining some options in mind:
>
> (1) Create a *generic topic* `unit-status` and make services consume and produce messages on this topic. [...]
>
> (2) Create a *specific topic for each status*, for example `unit-status-created`, `unit-status-packaged`, `unit-status-loaded`, `unit-status-deleted`, and so on.
>
> In sequence, the practitioner explains the drawbacks of each:
>
> (1) This has the consequence that *you consume your own messages and have to filter them.*
> (2) Requires a *code or configuration change* in potentially all service when a new status topic is added.

In another question [215], a similar inquiry is presented where a practitioner wonders about two event design approaches:

> (1) *A* sends a message which contains event and related entity id like: `entityCreated {entityID: 1234}`. *B* consumes this message and *if it needs further information, it fetches this from A* with `entityID`.
>
> (2) The message not only contains the information above, but *also metadata* like: `entityCreated {entityID: 1234, SomeFieldKey: someFieldValue, ...}`.

The practitioner continues addressing the pros and cons of each and wonders which one to choose:

> (1) Pros: *Less network usage*; *Always the same structure of messages.* Cons: If information from A is needed on demand there *must be some mechanism to catch*, e.g., network failures.
>
> (2) Pro: *Information is already there.* Con: What if the *attached information is not enough*?

Although practitioners seem aware of some of the consequences different event modeling options bring to the application design, the excessive number of questions, 19.71% of the total, suggests developers encounter uncertainties in deciding on an optimal event design.

Cause. As microservices are designed, implemented, deployed, and fail in an uncoordinated manner, asserting about formal properties of events and data schema of individual microservices in a unified manner constitutes an open problem.

Discussion. Differently from relational model [38], which provides a formal foundation that allows database designers to reason about the possible performance implications of a data model (e.g., the level of normalization dictates whether additional `JOIN` operations are necessary to retrieve certain information from the database), event design is still an open problem. As a result, developers navigate through many possible event designs that, if not carefully thought out, may impact the design of the application and the performance of operations asynchronously triggered by events.

In terms of best practices, event storming [22] technique can support development teams to establish boundaries, design, and assign events across microservices considering an EDA. The process facilitates identifying key events that are supposed to be exchanged across microservices and exposed to external systems. Despite being often referred to in industry settings [114], we have not found substantial evidence of event storming usage in the analyzed dataset.

Other industry best practices [48, 191] relate to the inclusion of metadata relevant to the event, denormalizing frequently accessed fields, modeling for data redundancy to avoid additional overhead on data access, normalization of infrequently updated data, and the reasoning about the interaction between components to determine the appropriate granularity of events and identify dependencies between events prior to system design.

5.2.2 Evolving Event Schemas (CEM2). We also observe that some microservice practitioners share uncertainties on how to deal with the possible effects of event schema evolution properly. In particular, as events are often parsed into microservices' own data models, practitioners express concerns over the impact of event schema changes on their microservice states. For instance, upon a new attribute being incorporated into an event's schema (user's `address`), a developer wonders how to update a microservice state retroactively [86]:

> (DEV#1) I can update my event and *now include the* `address`, but it will only work for new users, *the old ones will have* null *addresses*. Should I scan the whole database and manually dispatch an event for each user?

On the other hand, although events are considered abstractions to decouple microservices, some practitioners report event schema evolution impacting dependent microservices [160]:

> (DEV#2) services are tightly coupled by [event] schema and *can causes errors and this contradicts with event sourcing goal.* How can we address this problem?

Evolving event schemas also appear as an important challenge, corresponding to 20.59% of the questions about managing event schemas in microservices and accounts for 5.02% of the total questions.

Cause. Uncoordinated event and data schema changes in individual microservices can affect local and global properties of the system.

Discussion. We observe that developers are aware of the risks associated with schema changes and their potential impact on microservices dependent on the modified event type. If the type of incoming events does not match the expected event type in a consumer microservice, the processing of this event may fail. However, developers have not found a systematic solution to either minimize the impact of or comprehensively manage the evolution of event schemas across the network of dependent microservices.

The considered best practice in the state-of-the-practice is relying on a centralized repository for managing and validating schemas, such as Schema Registry [42], which relies upon data serialization

formats like Apache Avro [65]. However, Schema Registry requires a global adoption across all microservices, forcing every event producer and consumer to employ schema and data contract enforcement. Besides, adopting such tools is not always possible due to the lack of support for messaging technology adopted (as it only supports Kafka at the time this manuscript is written). Overall, this remains an open challenge in practice. Besides, the use of event metadata repositories has been gaining traction recently. Tools such as event catalog [62] can provide a unified metadata storage to map all events modeled, as well as their corresponding schemas, ownership, and historical changes, allowing development teams to reason about the created events to avoid errors, misleading assumptions, and duplicate events.

Finding 5. Developers find no principled ways to model events accounting for performance and decoupling tradeoffs. Systematic and technology-agnostic support for managing event schema changes is also an open problem.

5.2.3 Implications. With the lack of automatic support for event schema evolution, microservice developers can isolate failures caused by event schema mismatches and log them appropriately for later reconciliation. Messaging technology providers can supplement their documentation with more technical examples of how developers can isolate the microservices' application code from the impact of event schema changes. Guidelines on appropriately rolling out event schema changes without impacting individual microservices in the context of their technologies can also benefit developers [146]. In this line, a recent work [147] proposes, through framework support, a set of guidelines to support the data conversion process for schema evolution in event sourcing systems.

For modeling event schemas, researchers can develop analysis tools that holistically map the microservice event topology, matching producer and consumer microservices through the events exchanged. The tool can incorporate metrics to aid developers in reasoning about the tradeoffs of different event designs. Furthermore, researchers can provide a formal foundation for modeling events in microservice applications. The model must allow for reasoning about the performance tradeoffs of including specific attributes in events. Lastly, automatic support for event schema evolution across microservices is an open and complex problem that researchers could tackle.

5.3 Performance

Performance is systematically cited as an important factor for adopting microservice architectures [114, 137, 154]. Developers seek to reap the benefits of having the application decomposed into independently scalable building blocks [115]. In an architecture based on events, though, the performance of individual microservices is tightly coupled with their capacity to process and dispatch application-generated events at a rate that does not compromise the expected performance of the downstream microservices in the event flow. As a result, disruptions in the event flow can lead to critical performance impacts. In this section, we analyze such cases through the lens of microservice developers.

5.3.1 Event-Processing Overhead (CP1). Developers report particular event processing scenarios that ultimately lead to a performance penalty. For example, in connection with *CAS5*, some practitioners report the need to replay events to fulfill a given requirement, but that ends up introducing additional load to the application [34]:

> The validation of a *reservation request needs to know the previous and following reservations* (within 3 hours from the booking time of the incoming request).

> *If I ask for a reservation for tomorrow, the system will replay reservations from 6 months ago* that usually are not related with the incoming request.
>
> The developer then highlights the overhead:
>
> This leads to inefficiencies over time as result of the *huge amount of unnecessary events that are replayed.* I thought to solve it using daily snapshots but it seems the wrong way to do it.

In a similar way, in connection with CAS3, other developers acknowledge there could be a performance penalty on leveraging events for replicating data across microservices. For example [125]:

> The *Invoices* microservice must listen to all `ContactCreated` and `ContactDeleted` events in order to know if the given recipient id is valid.
>
> Then *I'd have thousands of Contacts* within the *Invoices* microservice, *even if I know that only a few of them will ever receive an* `Invoice`. Is there any best practice to handle those scenarios?

This challenge is the most prominent in the `Performance` category (53.57%), and spans 5.38% of the total questions. This suggests that microservices may execute with suboptimal performance in real-world deployments.

Cause. Developers are often able to identify the root cause of performance issues derived from event-based communication and computation. However, they express uncertainties in dealing with the overhead of processing events and their possible performance impacts.

Discussion. At first sight, the overhead of the first case can be potentially mitigated by storing the already events processed in the producer's own private state or cached. However, this practice introduces hidden dangers. Despite introducing a prohibitive storage overhead, this practice overloads the microservice with the responsibility of streaming events to consumers and ensuring no events are missing, tasks that should be part of the message broker. Most importantly, by favoring the data shipping paradigm [24], on which applications offload the application state to an external system (e.g., database system, message broker, or cloud object storage), storing events in a microservice's local state would contradict the paradigm, thus increasing the complexity of the microservice. Similar to CAS5, the explicit management of event log offsets offered by event log systems such as Kafka is considered a good practice to avoid the overhead of streaming useless events as part of a consumption request.

In the second case, developers could benefit from filtering event streams. However, this may not be possible in case the filtering criteria cannot be extracted from the event payload (e.g., which contacts contain an invoice). In this case, it would be necessary to correlate the event with some data belonging to a specific microservice private state, creating challenges due to the state encapsulation property.

5.3.2 Large Event Payload (CP2). We observe developers also present concerns over the size of the event payload generated and consumed by different microservices. For instance, a common concern lies in whether publishing events with large payloads into the message layer entails a good practice:

> (DEV#1) My question is, is it wise to stream such file over event bus from Service A to API gateway? (*File may get as large as* 100 MB) [163]
>
> (DEV#2) If an event needs to pass a *very large volume of data* to the next Saga event, how is this done in terms of the request structure? Is it divided into multiple Sagas for example (as a result pagination type)? [75]

(DEV#3) I think we will need to go with second option, i.e., not sending data in the event due to the *potential size of the event data*. [170]

To circumvent the potential performance degradation incurred by processing large event payloads, some developers look for alternatives to the solo event-driven approach, but end up encountering other issues:

(DEV#3) I was also thinking about adding the URI in the event so that the service just needs to call it. Only issue I can see is that *how would the service know what type to deserialize* the response to? [170]

The performance impacts of generating, processing, and storing large event payloads account for 17.86% of `Performance` challenges. This challenge represents 1.79% of the total questions and highlights that developers find difficulties in dealing with large event payloads and their consequences for microservice performance.

Cause. By assuming misleading assumptions about the storage and system performance constraints of state-of-the-art messaging technologies, developers end up facing issues in achieving application requirements related to transferring large amounts of data across microservices.

Discussion. Message brokers and modern log-processing systems are not designed to handle messages with large payloads natively. Message brokers have historically targeted systems' integration cases, which usually do not require large event payloads [154]. Log-processing systems, like Kafka, for example, were initially designed for the timely processing of logs extracted from processes running in distributed servers [111]. Shared log systems like Kafka allow payload compression, which comes at the cost of higher CPU utilization, jeopardizing core event processing. Similarly to message brokers, though, it was never a design goal to accommodate the processing of large log payloads. Until recently, Amazon SQS only allowed event payloads up to 256 KiB [10]. Similar limits are encountered in other messaging systems. Native support for these types of objects is found in cloud storage services and specific column-oriented database systems, like `BLOB` type in PostgreSQL [151]. The discussion of the questions analyzed about this problem scope suggests, as best practices, that microservice developers may benefit from either redesigning their computations to manage large objects in appropriate storage systems.

Large payload events can potentially be stored outside the MOM, in a standalone object store or database. However, the additional points of failure and components to support, the rapid growth of external storage, and the incurred latency caused by persisting and retrieving payloads on external storage are often mentioned as drawbacks of this practice [17].

Developers may also rethink the granularity of their event payloads when they grow arbitrarily. This may involve breaking down application components into smaller, finer-grained tasks to decrease the size of events transmitted across microservices.

5.3.3 Fluctuating Event Rate (CP3). Developers describe challenges associated with dynamic workloads and their impact on the observed event processing rate.

(DEV#1) In case of *pressure over the system*, can service B communicate to service A to *slow down*, and A will react to this by *not accepting more requests* from clients, till B decides it can continue? Is this something that can be achievable using [(omitted library)]? [159]

(DEV#2) I want to *restrict the users from publishing a new message* on this topic to *prevent* my system from *choking after a certain limit*.

A developer proposes to impede a certain producer from appending new events to a Kafka topic:

> For example, if the number of unacknowledged messages in the topic is already *more than or equal to 10,000*, then I want to give a bad input *exception* or something to restrict users from flooding my queue [55].

Another developer notes an increased network traffic over time:

> (DEV#3) *Traffic is increasing* and we've noticed that events *spend a lot of time in queue*. We need to *process events faster*.
>
> Payload for each event is small and this solution has been working for some years now and a couple of years ago they thought that having a queue was a good idea but now I'm having this performance issue. [...] Is it possible to implement circuit breaker or a failover mechanism for an async call? [107]

Lastly, a developer expresses concern on how to handle scalability in consumer microservices:

> (DEV#4) Microservices allow an elastic number of instances which are scaled up and down automatically based on the current load. This characteristic should not be limited by Kafka.
>
> We need to guarantee an at-least-once delivery We need to guarantee the delivery order of events concerning the same entity.
>
> *How is scalability handled here? The amount of partitions shouldn't limit the amount of consumers, or does it*? [...] Is there a better way to guarantee the goals mentioned above? [174].

This performance challenge strengthens our observation that microservice developers encounter difficulties on reaching performance goals in event-driven microservice architectures. The challenges about adapting microservices to handle increased influx of events represents 28.57% of `Performance` questions and represent 2.87% of the total of questions.

Cause. Developers are presented with challenges involving the adaptation of microservices' performance to stringent workload scenarios. As highlighted above, it is neither crystal clear for developers whether traditional approaches to circumvent increased load in web servers can be directly applied to asynchronous event processing in microservices, nor how to optimize default messaging technologies primitives in pair with application semantics in order to prevent consumers and queues from becoming overloaded.

Discussion. With traditional synchronous communication paradigms, like RPC and HTTP requests, an application can, for example, define a threshold of the maximum allowed concurrent connections, which can mitigate performance issues in some cases. In event-based architectures, though, there is an indirection: the events are often stored in the event management layer first and later forwarded to (or pulled by, depending on the message system) consumers. This decoupling introduces challenges for consumer microservices to communicate whether producers must decrease the event generation rate or even refrain from generating new events. On the other hand, modern event management systems, like Kafka, provide abstractions to fine-tune the consumption rate through, for example, adjusting the amount of bytes pulled from Kafka brokers and to parallelize the consumption of events through partitions. However, as discussed in CAS4, developers must explicitly configure both the consumption rate and the unit of parallelism according to their application semantics, performance requirements, and resource constraints, which may not be trivial for newcomers. Container technologies such as Docker and Kubernetes allow for configuring scaling up properties based on system thresholds such as CPU and memory consumption but are oblivious to queuing properties and roles, often requiring manual configuration from application maintainers.

Besides controlling the consumption rate, practitioners can resort to adjusting the push rate of events at the producer side. Web frameworks like node.js streams [67] and javascript streams API [136] and reactive libraries like RxJS [44], RxJava [43], and RSocket [45] are popular alternatives. However, care must be taken on buffering events at the producer side since crashes and over-usage of memory space can lead to losing buffered events. Therefore, developers must consider the correctness and performance tradeoffs of application-level event processing rate approaches.

Finding 6. Developers encounter challenges on managing the event rate across producers and consumers. This only exacerbates in the presence of event replays and large event payload sizes.

5.3.4 Implications. Message technology providers can provide efficient and proactive event-processing techniques and algorithms that mitigate or decrease the impact of fast producers and slow consumers. As mentioned in the discussion of individual challenges, developers may benefit from rethinking the granularity of their microservices and the content included in their event payloads to avoid adding overhead to their event processing pipelines. Message technology providers can additionally allow for queuing higher message payloads as done recently in Amazon SQS [10], which can potentially mitigate many use cases that process large amounts of data. Other mechanisms that can aid is offering additional network protocols that can better accommodate large content payloads, such as QUIC [96]. Developers may also be aware that, as events flow through microservices, the events generated by each microservice may include additional data. That suggests developers may track the event payload sizes more carefully as they flow across possibly many microservices.

Researchers can investigate whether state-of-the-art approaches for scaling distributed systems also apply to event management in microservices. Besides, researchers can develop systematic tools to mitigate the effects of large payload sizes in event-based microservice workflows. That could involve optimal deserialization of event payloads (e.g., only deserializing the subset of attributes that the microservice requires), advanced compression tools that mitigate the overhead of growing events, and transparent removal of unused event attributes.

5.4 Observability

Observability is a critical concern in modern software development because it allows developers to track important application behavior and metrics, such as application exceptions and memory usage, respectively [19]. In distributed systems, such as microservice architectures, observability becomes more critical since problems can possibly span across multiple components that execute independently in a decentralized manner [19, 200].

In our analysis, we were surprised to find that microservice developers leverage events not only as an abstraction to process operations asynchronously but also to track the progress of operations across multiple microservices and troubleshoot the complex interplay of multiple microservices. In this section, we describe these patterns and their associated challenges.

5.4.1 Event Flow Observability (CO1). Although tools to observe synchronous requests (HTTP or RPC-based) crossing microservices enjoy apparent consolidation, developers express challenges in monitoring the asynchronous event flow across microservices, as exemplified by the two following developers' quotes:

(DEV#1) What is something we can do to have an *end to end understanding* from producers to consumers, etc? Mainly for troubleshooting purposes/change management [54].

(DEV#2) What is the best way *to track the JSON as it flows* through many microservices down stream in an event-driven way? [171]

Another example [8] presents a developer wondering whether an observability tool can embrace events:

(DEV#3) Will [(omitted tool)] *be able to trace requests done over the eventbus*? The tracing page says that headers need to be propagated through in http or grpc—but the eventbus sends messages via tcp—does that mean that [(omitted tool)] will not be able to trace requests and show the visualisation tools[?]

Furthermore, we found that issues in the message broker only exacerbate the challenge of monitoring how healthy the event flow is, as explained by another developer [187]:

(DEV#4) For any reason *the messages get stuck after some time [...] There is nothing in the log files nor the OS event log.* I have to restart the ([omitted technology]) service in order to "reanimate" it.

Afterwards[,] all stuck messages will be processed and *everything is working fine until the next "accident."* [...] Does anybody has an idea what I could check additionally to find out what the problem is?

Observing the event flow appears significantly in *Observability* challenges (89.47%). This highlights the difficulties developers face in understanding the progress of events as they flow across microservices.

Cause. Application-generated events serve as natural progress markers since they serve as the unit of representation of actions that form business transactions that span across microservices. Therefore, it is a natural choice for developers to obtain an end-to-end overview of the application behavior based on these events. The concurrent nature of transactions, however, leads event histories to grow, interleave, and exhibit discrepancies arbitrarily, rendering difficulties in identifying the source cause of problems and matching correlated events to aid troubleshooting.

Discussion. Microservice developers often rely on specific-purpose observability platforms such as Datadog [53], Dynatrace [122], Grafana [112], and NewRelic [98], to centralize all application logs. Although some of these solutions offer open source features, it is common for microservice adopters to acquire their cloud services, offloading the provisioning of computational resources to the platform instead of explicitly managing CPU, memory, and storage for the observability of microservice applications. However, developers encounter a twofold problem. On the one hand, developers find that state-of-the-practice observability platforms do not natively support tracking application-generated events. Developers are in charge of explicitly pushing application-generated events to observability platforms, creating challenges similar to those discussed in CAS1. Most importantly, failures in process can lead to gaps in the event log, complicating the comprehension of the application behavior. On the other hand, when disruptions in the event flow are not caused by crashes or bugs in the microservices themselves, but in MOM, developers have difficulties in finding the root problem. Although MOM are only part of the intricate and heterogeneous components in microservice deployments [115], they are a core enabler of event-based interactions in microservices. Thus, timely detection of message broker failures and their relationships with microservice disruptions is key to observability in event-based architectures.

5.4.2 Auditing via Event Replay (CO2). In CAS5, we discussed how developers leverage events in an attempt to synchronize data across microservices. As a related pattern, we also found that developers use the events as an abstraction to reproduce some application behavior. Similar to

CAS5, developers end up facing difficulties in ensuring the correctness of the process, as explained by a developer [189]:

> (DEV#1) If the VisitId was deleted across all services we could just *replay the events* one at a time, in order, and reproduce an exact copy of the original record.

The developer then mentions an impedance commonly expressed by another practitioner across the questions:

> I've been using [(omitted message technology)] for the stream itself [...] The issue though is that *this is not suitable for a replay store*—only delivery of the event messages.

Another developer warns about the hidden dangers of such a method [205]:

> (DEV#2) What's maybe not a good idea is trying to *recover the current state* of your domain model by replaying an arbitrary set of your events.
>
> Remember, *an event isn't usually a complete representation of the state of the model after the change*, but rather a description of the things that changed. (..)

Auditing past microservice executions through events to observe and further understand their behavior appears in 10.53% of `Observability` challenges. This suggests that current technologies offer insufficient abstractions to support event replay effectively.

Cause. As events often represent intermediate microservice states, they appear as convenient abstractions to developers seeking to troubleshoot past application behavior. However, in the same vein as CAS5, this practice incurs hidden challenges due to the lack of appropriate support from current messaging and framework technologies. Developers must end up building customized solutions and environments for safe resubmission and/or reprocessing of events, accounting for their possible effects in private microservice states, a complex and error-prone task.

Discussion. As discussed in CAS7, mismanaging the processing of duplicate events and their possible effects on the application can lead to corrupting a microservice state. Another example, based on CEM1, is that microservices may even fail to replay events due to the divergence of event schemas.

To the best of our knowledge, although there is no considered best practice, application maintainers can adopt techniques from the software engineering testing field by, for example, putting forth multiple testing environments that resemble the production environment [153]. Developers could maintain a staging deployment to proceed with auditing, possibly encompassing only the key microservices involved in order to decrease complexity and costs. Thereby, preventing unpredictable and undesirable effects on the production environment.

> *Finding 7.* Developers find difficulties in observing and thus reacting to disruptions that occur in the event flow across microservices. These disruptions may not only be caused by microservices, but also by message brokers and network partitions.

5.4.3 Implications. MOM providers can evolve their systems and APIs to provide native integration with industry-strength observability tools. The integration must account for the needs of developers in their real-world cases as discussed above, including but not limited to specific metrics such as event processing and acknowledgment delays from consumers.

For developers, as state-of-the-art abstractions render limited support for safely replaying events, developers can use a "staging" environment (similar to testing environments with testing, staging,

and production [197]) to audit their event-triggered operations. However, generating an application and message broker state from a previous point in time for replay can be challenging.

In the tool development landscape, opportunities for researchers are: (i) They can co-design static analysis tools and dynamic analysis of event streams for a holistic monitoring of microservice event streams. In particular, this can aid developers by automatically matching correlated events and associated service disruptions in the event flow. (ii) They can develop techniques for isolating the re-execution of events from impacting the actual microservice states. Recent tools [119] allow for replaying operations in database-backed applications. However, it is unclear how the approach can be applied to events and distributed states of microservices.

5.5 Security

Security is another critical concern in event management in microservices. In this section, we discuss developers' requirements and challenges when ensuring that event ingestion, processing, and storage meet security constraints.

5.5.1 Authentication (CS1). Although events are often an abstraction that is only internally recognized by the microservices, some events may be generated by end users. That requires making public APIs available, which necessarily exposes the microservice to security vulnerabilities, as exemplified as follows:

> (DEV#1) Because I *don't check authorization* on websocket open, in theory this approach is vulnerable to a *dDos attack*, where an attacker simply opens as many sockets as they can [30].

On the other hand, even when APIs are safeguarded with authentication methods, developers are concerned about aligning the validity of the authentication mechanism with the event flow.

> (DEV#2) In case of event-driven application, it's possible *to ensure the token is valid*? In case of failure, the user clicks on button and an event is written but the processing of this will be hours later. How can I process the event with the user credentials? [128]

> (DEV#3) if I change my communication mechanism to using *async messaging* (e.g., RabbitMQ), how would I now authenticate the event to the scope of the user who initiated the event [194].

Challenges in ensuring event-based workflows and event-triggered computations execute securely appear substantially in `Security` challenges (76.92%). Spanning across 3.58% of the total questions, this suggests developers fall short on a holistic solution to secure their event-driven microservices.

Cause. Developers seeking to ensure secure parties are authenticated in event-driven microservices must align authorization mechanisms and event processing systems. A key obstacle is that authorization and messaging mechanisms are often provided by different provider/systems, resulting in a system integration issue.

Discussion. Developers working with microservices need to handle authentication token lifecycles and make sure that any events related to the token are in line with its validity. Despite being error-prone, this process can add complexity to the solution. Preferably, developers must seek the use of broker-provided mechanisms for security in favor of external mechanisms. In this line, NServiceBus [123] and Azure Service Bus [131] offer declarative access control through specific IP addresses or ranges. In addition, a good practice in confidential computing is ensuring that the application execution is protected in an isolated environment [11]. For instance, enclave nodes could be leveraged to host specific brokers that contain privacy topics/queues, which can also be secured over specific security protocols. However, confidential computing often involves performance

tradeoffs, as there can be an overhead on securing both the parties involved in communication and the communication link itself.

5.5.2 Data Privacy (CS2). Data privacy emerged as a key requirement in modern applications to prevent leakage of user data and potential violation of data privacy laws such as the General Data Protection Regulation (GDPR) [60]. Data privacy laws prescribe that users can request the removal of their data at any moment, and firms are usually given a deadline for fulfilling the request. As discussed earlier, in microservices, data flows from the multiple event streams to many microservice consumers, which in turn apply operations on their private states. This heterogeneous data placement and movement can make managing data privacy challenging. As exemplified by the following quote [124], we observe that microservice developers express uncertainties about handling data privacy in event management.

> *deleting user data* in an event-sourced system [...] *To which stream is the tombstone event written*? The event stream of the specific user? Or is there an event stream specifically for tombstone events?
>
> *To keep some data* (e.g., the users' id), he advises to split the user stream into a *public and private event stream.* [...] How can you make sure that both events have been appended?

The developer continues describing the problem, referring to another challenge already discussed in this work about the existence of dependence between events:

> Let's say a command needs to append an event to both the public and private user stream. How can you make sure that both events have been appended? Does the event store publish both, *SomeUserEventPrivate* and *SomeUserEventPublic*, to the event bus? (57784098).

Managing data privacy in the context of event streams accounts for 23.08% of security challenges. This suggests that developers find uncertainties on how to properly manage data privacy in events exchanged between microservices.

Cause. The lack of out-of-the-box privacy insurance features in state-of-the-art messaging technologies renders challenges in anonymizing, encrypting, controlling access, and deleting user data for confidentiality in event-driven microservice architectures.

Discussion. We find that developers often manage data privacy in event streams using ad-hoc solutions, such as separating privacy and non-privacy data into different streams. However, these mechanisms do not enforce, by design, that data privacy issues are prevented. For instance, a user's data contained in events that are not supposed to be visible to certain microservices can still be freely consumed by any microservice or any system external to the microservice architecture. Besides, distributing events into different topics can later lead to consistency problems. As observed above, the developer has to explicitly ensure that privacy and non-privacy streams are in sync.

Developers can separate topics into brokers that are safeguarded by conventional secure access protocols. Despite Kafka obliviousness to secure access protocols and the potential performance overhead, such an approach can possibly mitigate some privacy issues. Another considered best practice is using naming conventions to represent internal or sensitive data streams so as to facilitate access control [20]. Besides, data minimization approaches can be employed, such as masking, tokenization, and pseudonymization. For instance, developers can mask sensitive data in each node but must account for data dependencies that require unveiling masked data in every consumer.

Finding 8. It is unclear for microservice developers how to properly safeguard security properties in the context of event processing in microservices.

5.5.3 Implications. Messaging technology providers could provide technology-specific guidelines on properly handling authentication and data privacy in their systems, accounting for the mechanisms that prevent exposing unintended event payloads. Cloud providers and framework developers can offer custom deployment templates to facilitate meeting certain security criteria. However, these should not be oblivious to the event management layer, suggesting that an appropriate cross-system integration is necessary to properly fulfill security challenges.

Microservice developers can, by principle, never include sensitive data in their generated events (i.e., privacy-by-design). However, this can jeopardize the decoupling benefit brought by event-based architecture, requiring consumers to contact the producer for the missing sensitive data. Researchers can extend existing code analysis tools to alert developers about bugs and configuration mistakes that can lead to unintended access to events and exposure of private data in event streams. Besides, researchers can investigate privacy-by-design event management methods without impact decoupling. For instance, protocols that automatically encrypt sensitive data in events without developer intervention.

Summary. Microservice developers encounter a myriad of problems when managing events, including large event payloads, evolving event schemas, auditing applications through events, enforcing privacy in events, and processing event dependencies and out-of-order events.

5.6 Synthesis of Results

In this section, we summarize the key takeaways of our findings. In particular, we distinguish foundational challenges, that is, problems inherent to event-driven microservice architectures, from a lack of engineering mastery from developers, as summarized in Table 7.

5.6.1 Foundational. In roll-backing states (CAS2), messaging systems are not aware of encapsulated application states, and database systems (or any data store) that store application states are unaware of events. Thereby, developers must explicitly synchronize data in heterogeneous systems to achieve application safety, a challenging task.

To process event dependencies (CAS3), developers must explicitly track the causality of events across distinct event streams, besides requiring the encoding of specific rules governing how the causality relation is established between events, challenges inherent to asynchronous systems such as weak delivery semantics pose complexities on ensuring the correct processing of event dependencies.

In synchronizing microservice states via event replay (CAS5), similarly to CAS2, by requiring explicit coordination between the messaging system and disparate microservice states, the task of ensuring application safety is the responsibility of developers. In particular, republishing and subsequently reprocessing events arbitrarily can lead to corrupting microservice states and a prohibitive cost on performance.

Developers benefit from the application data that is often embedded in events to build a view of the application state. We observe the incidence of leveraging events to build views in CAS6. The difficulties expressed by developers, the ad-hoc approaches put forth, and the disdain for stream processing engines suggest that streaming systems could evolve to provide a better programming interface and integration to developers to better meet their expectations.

Table 7. Overview of the Challenges

Category	ID	Challenge	%	Problem Class
Safety and Liveness	CAS1	Publishing events safely	15.13%	Lack of mastery
	CAS2	Roll-backing states	11.18%	Foundational
	CAS3	Processing event dependencies	18.42%	Foundational
	CAS4	Processing events in order	16.45%	Lack of mastery
	CAS5	Synchronizing states via event replay	15.79%	Foundational
	CAS6	Continuous queries over events	9.21%	Foundational
	CAS7	Weak delivery semantics	13.82%	Lack of mastery
Event Schema Management	CEM1	Modeling event schemas	83.87%	Foundational
	CEM2	Evolving event schemas	16.13%	Lack of mastery
Performance	CP1	Event-processing overhead	48.15%	Lack of mastery
	CP2	Large event payload	29.63%	Lack of mastery
	CP3	Fluctuating event rate	22.22%	Foundational
Observability	CO1	Event flow observability	87.5%	Lack of mastery
	CO2	Auditing via event replay	12.5%	Foundational
Security	CS1	Authentication	81.82%	Lack of mastery
	CS2	Data privacy	18.18%	Foundational

Developers miss a formal model when modeling events for distributed operations in microservices (CEM1). That would allow practitioners to reason about application safety and performance requirements in their system design.

To maintain the loose-coupleness prescribed by EDAs, microservices are often deployed via autonomous processes that communicate via asynchronous events. However, the disparate application logic and workload encountered in each microservice and their interrelation through event processing can affect downstream microservices (CP3). In particular, fluctuating event rates render difficulties for developers, which requires customizing performance configuration strategies for message channels, aligning with the particularities of each microservice.

Auditing transactional workflows via event replay (CO2) is performed via ad hoc processes that require developers to provision resources and coordinate events and state updates in a way that does not compromise application safety and liveness, which is largely error-prone. There is limited support for auditing event-based workflows in microservice architectures today.

Handling data privacy in EDAs requires a combination of design choices, data governance policies, and application encodings (CS2). It is the responsibility of microservice developers today to reason about privacy policy declaration and correctness, and handle end-to-end processing and enforcement of privacy policies homogeneously across disparate microservices.

5.6.2 Lack of Mastery. Although developers may face tradeoffs related to added architectural complexity, mechanisms such as change data capture can mitigate some of the challenges discussed in CAS1 by removing the event publishing as part of the critical path of local transactions. However, care must be taken with the eventual delivery guarantee. We also observe that this challenge arises due to insufficient comprehension of message broker APIs and guarantees. For example, for publishing multiple events atomically as part of an operation, event systems like Kafka already offer transactional appends across different topics, guarantees often overlooked by developers.

For processing events in order (CAS4), microservice developers must be mindful of the expected processing semantics of producers and consumers of the message technology. In weak delivery

semantics (CAS7), it is required of developers to coordinate offset acknowledgments and application state, preventing consumers from processing events arbitrarily. However, idempotent code may not be possible to implement in some cases.

Despite the lack of automatic solutions for evolving event schemas (CEM2), existing tools offer solutions with reasonable tradeoffs, with a technology-specific mechanism still an open problem. Challenges related to event-processing overhead (CP1) and large event payload (CP2) mostly occur due to mismanagement of message queues and topics. In event flow observability (CO1), we observe developers express difficulties in integrating application-generated events with state-of-practice observability tools, characterizing a lack of expertise in utilizing these tools properly. Out-of-the-box solutions for ensuring only authenticated parties exchange events (CS1) can be incorporated into EDAs today.

6 Threats to Validity

In this section, we discuss the threats to the validity of our empirical study.

Selection Bias of the Source. Similar to previous research [13, 213, 216], our work uses SO as the data source to study the challenges developers encounter in practice. Although other data sources could supplement the findings, the heterogeneity of challenges, application scenarios, and developer expertise found in SO enables a trustworthy reflection of the state of the practice. For instance, the findings are aligned with several blog posts that report similar challenges [32, 59, 81, 142, 143, 175, 176, 179, 180, 217].

Construction of Tag Set. We select a set of tags to filter SO questions associated with event management in microservice architectures. The relevant keywords and metrics applied may omit some tags associated with event management in microservices. To mitigate this threat, we rely on manual inspection conducted by independent researchers, and we adopt the lowest threshold used in previous work [13, 213, 216] to include the highest amount of tags as possible.

Subjectivity of researchers. Since we adopt manual analysis to identify and categorize challenges, this may threaten our findings' validity. To minimize this threat, the first two authors analyzed and classified each filtered question in isolation. Upon conflicts, an experienced arbitrator (last author) coordinated the discussion to reach an agreement.

Generalizability of Our Findings. We recognize that the application scenarios, requirements, design choices, event processing patterns, and the challenges extracted from SO may not be generalizable to all possible cloud and data platforms to which event-driven microservices can be deployed. However, as SO is an industry-strength questions and answers platform for developers, their questions tend to shed light on the most adopted industrial practices. Thus, the 10-year question range covered in this study strengthens our perception that the findings highlight a substantial portion of the challenges in the state of the practice.

7 Related Work

In this section, we discuss related work about investigating challenges that developers face in microservice architectures and research lines that intersect with the challenges studied in this work.

7.1 Empirical Studies

Benefits of Microservice Adoption. Zhang et al. [219, 220] conduct interviews with microservice practitioners to understand the benefits brought about by microservice adoption. They find organizational transformation, decomposition, distributed monitoring, and bug localization as prominent challenges. Wang et al. [208] interview and survey practitioners to collect and categorize best practices, challenges, and their related successful solutions employed by practitioners. They find

managing API changes, a lack of support for monitoring, and finding microservice granularity as common challenges.

Microservice Issues. Zhou et al. [221] performed a survey to characterize typical faults, debugging practices, and the challenges entailed by troubleshooting failures in microservice architectures. They find microservice developers can benefit from improved trace visualization tools, specially in cases related to microservice interactions. Ramírez et al. [162] mine SO posts to identify common issues on development and testing microservice applications. They find missing parameters as a prominent issue in communicating with other microservices, wrong library versions as the main impediment for service discovery, and connecting to other microservices with correct user credentials as an example of an authentication and authorization challenge. Waseem et al. [209] employ a mixed-method empirical study to understand the types of issues microservice developers experience. Similar to Ramírez et al. [162], they find programming errors, missing artifacts, invalid configuration, and communication as the main causes behind the issues.

Data Management in Microservices. Laigner et al. [115] characterize data management challenges in microservice architectures through a mixed-method empirical study. They focus on analyzing developers' pitfalls when implementing data management logic in the application layer and discussing the limitations of state-of-the-art database systems that prevent them from better serving microservice architectures.

Event Sourcing Pattern. Overeem et al. [147] study the challenges practitioners experience using event sourcing. Through interviews with 25 engineers, they find event system evolution, steep learning curve, lack of available technology, rebuilding projections, and data privacy as the main issues affecting the development of systems that employ the event sourcing pattern. Of the 19 systems mentioned by interviewees, only 8 apply the microservice architectural style.

Security in Microservices. Nasab et al. [138] perform a mixed-method empirical study to understand the security practices in microservice systems. They collect 28 security practices from open source repositories and SO posts, which were later confirmed through a survey with practitioners.

Exploratory Studies in Microservices. Hacaloglu and Demirors [84] study the usefulness of events for software size measurement. They find preliminary evidence that events can supplement software size measurement techniques. Lazzari and Farias [63] report an exploratory study that compares event-driven and REST architectural styles in the context of modularity. They find preliminary evidence that EDA improves the separation of concerns.

Overall, we observe that challenges related to event management are overlooked in the literature. For instance, although few works report developers mentioning possible issues with message technology systems [138, 209] and asynchronous task invocation [162, 221], they are often addressed as general microservice configuration or invocation problems, preventing a proper characterization of the problem. Therefore, although event management in microservices has been rapidly gaining industry popularity [32, 59, 81, 115, 142, 143, 175, 176, 179, 180, 217], it has not received due attention from the research community. In this work, we make the first attempt to investigate the specific challenges that developers face when managing events in microservice architectures.

7.2 Research Lines

7.2.1 Safety and Liveness. Safety and liveness have been historically a subject of study from the systems and programming languages communities. As highlighted in our discussion (Section 5), some challenges identified exhibit resemblance with distributed transaction and query processing, coordination programming, choreographic programming, and session types. In the following, we present related work in the aforementioned topics and discuss the shortcomings of the state-of-the-art that prevent properly addressing safety and liveness in event-driven microservice architectures.

Multitier programming [211] is a paradigm to develop the components that spans across different tiers in the system (e.g., client, server, and database) in the same compilation unit. Multitier programming targets enhancing maintainability and program comprehension, and enabling formal reasoning about the properties of the distributed application as a whole [211]. They do so by generating correct code at runtime or relying on the compiler through code-centric methods such as user annotations, static analysis, and types. Despite the multitier characteristic of event-driven microservice architectures (message broker as a tier interconnecting multiple microservices), we have not found evidence of the use of multitier programming in this study. Industrial solutions related to the adoption of multitier programming often relates to user-facing services [211].

Choreographic programming refers to the definition of a concurrent system through a single compilation unit called choreography [134]. More specifically, a choreography defines a precise specification of the communication behavior of a system [133]. In contrast to the traditional approach of specifying the components and a separate set of APIs that defines the input and output messages of each, a choreography prescribes the specification of the expected communications among components in advance, forming a global description of the system. The specification is used to generate a correct implementation of each component, e.g., deadlock-free interactions.

Similarly, *synthesis of implementations from* ***message sequence charts*** **(MSCs)** [49, 50] can generate executable system components based on the properties specified in MSCs. MSCs are a formal visual language to express the interactions between components, including processes, messages, and ordering relations. MSCs appear to be particularly helpful in spotting errors early in the design, such as possible data races [113]. A commonly mentioned drawback of visual tools that model concurrency is diagram complexity, specially in large-scale and complex code bases, which can affect maintainability and execution flow understanding [166].

In the space of *general-purpose programming languages*, extensions that account for loose-coupleness and event-driven features are interesting to support developers to adhere to microservices requirements. We note particularly extensible extension languages like Lua [95], which offer varied language features that can be combined to build coordination abstractions for distributed asynchronous computing, namely, closures, coroutines, dynamic execution environments, and functions as first-class values [181]. Programming languages' features aside, general-purpose languages often offload coordination correctness to application developers. In this realm, Argus [121] is known for offering both a programming language and system to design and execute distributed programs. Although not having a modern implementation, Argus has influenced several developments over time, like middleware frameworks such as CORBA and Java RMI, object-oriented databases, and distributed object-oriented frameworks like Orleans (discussed below in *actor model*).

Despite the advantages of choreographies and the synthesis of implementations from MCSs, we have also not found evidence of the use of choreography languages like Chor [133] and related technologies, or tools and frameworks that support the MCS synthesis in this study.

We believe that tools and languages for multitier programming, choreographic programming, and MCSs synthesis could be leveraged to reason formally about some of the properties pursued by practitioners studied in this work. However, additional studies could survey practitioners' rationale and requirements in order to better understand the incurred adoption friction. We conjecture that the description of a global behavior of a system can be seen as dissonant to the decentralized, independent, highly dynamic nature of components and communication channels, and the decoupling of components pursued by practitioners in event-driven microservice architectures.

Session types are used to express and enforce correctness guarantees about communication in distributed systems, such as ensuring messages across concurrent processes are in order, comply with the expected types, and do not lead to deadlocks [94]. In this line, we highlight Effpi [173], a toolkit that provides a set of facilities to program and execute strongly typed concurrent, distributed,

and message-passing programs written in Dotty, the precursor of Scala 3. It includes an embedded DSL, extendable classes to define communication protocols as types, a compiler plugin for protocol conformance checking, and a runtime for performant execution. Effpi leverages session types to identify protocol violations, deadlocks, and livelocks before the application is deployed.

Coordination programming focuses on the interaction of the components from a higher-level perspective compared to choreographies, abstracting away from their internal functionalities/computations [78]. Rather than correct-by-construction communication, the coordination programming approach relies on extending a language to enable communication and coordination [212]. Coordination languages like Linda prescribe a shared memory system called tuple space, where data is stored as typed objects called tuples [29]. The tuple space is used for communication between processes.

Tuple space has been implemented in many industry-strength programming languages and runtimes like Java, .NET, Python, and Ruby. A popular Java implementation of tuple spaces is JavaSpaces [126], which treats Java objects as the record unit of a distributed system state. JavaSpaces target facilitating the implementation of distributed algorithms, such as Master-Worker pattern [190], by providing similar APIs as message queue systems. Other coordination languages like Bach [100] build upon the Linda model and present embeddings in Scala [145]. We note that tuple space has not arisen as a popular paradigm in modern distributed applications [184]. Despite the decoupling aspect provided by tuple space, in which processes do not need to refer explicitly to each other, the shared memory model is at odds with the principle of independent and encapsulated states in microservice architectures.

The *actor model* is a model for concurrent computation in distributed systems [2]. Actors model a small unit of computation with an encapsulated state and interact via message-passing semantics. To send a message, an actor must specify the target. A mail system buffers the messages and eventually delivers them. An actor accepts only a single message at a time and can perform local state changes, send a message, and create other actors. Concurrency in an actor system is enabled via pipelining and dynamic actor creation, being only constrained by the available hardware resources hand the logical dependence among actors.

Actor systems are realized either via a native programming language (e.g., Erlang) or an application framework (e.g., Akka for Java programs and Orleans for C# programs) support. In both cases, they posit their own communication mechanism, via actor message-passing, often realized by asynchronous RPCs. Although microservices can be implemented with an actor language or framework, by bypassing a MOM such as Kafka, the default communication mechanism of these systems excludes microservices that are not implemented as part of the same actor system.

Microservice developers could, in principle, forward actor messages to the MOM. Despite creating a duplicated message in the system, which creates the necessity of mapping external responses to system messages, by opting for such an approach, developers inherit challenges discussed in Section 5.1, such as CAS1, CAS2, CAS4, and CAS7. Similar reasoning applies to choreography, message-passing, multitier, and domain-specific languages.

Choreographies, session types, actor systems, and programs derived from multitier programming operate at or as part of the application process, including but not limited to ensuring messages are sent and received by the correct process, no undefined messages are processed, the order of messages is strictly followed, and no communication is left uncompleted. On the other hand, MOM leveraged by microservice architectures operate decoupled from concurrent processes (i.e., microservices), as a service, ensuring durability of messages even under failures, message delivery order, and providing high availability via fault-tolerance mechanisms, parallelism, load balancing, and supporting the dynamic creation and deprecation of message channels, producer and consumer roles. It is worth noting that these are orthogonal concerns, which can partially explain the popularity of MOM over

solutions from the systems and programming languages communities in event-driven microservice architectures.

Furthermore, while communication safety often appears as a pursued property in choreographic programming, message-passing languages, and session types, we observe that many challenges uncovered in this work relate to data-centric properties , such as all-or-nothing atomicity (CAS1), data invariants enforcement and distributed consistent states (CAS2), consistent data replication (CAS5), consistent queries (CAS6), and idempotence guarantees (CAS7), which requires reasoning about microservice encapsulated states and corresponding state updates triggered by events.

In this regard, Zuckmantel et al. [224] envision a formal semantic model to enforce transactional properties in event-based systems. The vision relies on three building blocks: (i) the extension to the traditional interface automaton model to capture the semantics of cross-microservice transactions, (ii) a coordinator process to control multiple instances of interface automaton representing concurrent transactions, and (iii) the use of dynamic runtime monitors to reason about causality constraints that span beyond the interface automaton lifecycle.

Incorporating transactional properties, in particular transaction isolation, into an event-driven system is promising due to the ability to combine event-based and data-centric properties. We note, though, that realizing such a vision requires that all transactions are expressed as interface automata, the definition of an expressive specification language that can refer to the timing and attributes of events, and the design of a horizontally scalable coordinator.

Solutions integrating multiparty session types with industry-strength data platforms that permit the execution of third-party applications also hold potential to bridge application correctness with fault-tolerant and durable MOM. In particular, Viering et al. [203] propose a framework that integrates multiparty session types with event-driven programming. The authors demonstrate the practicality of the solution by implementing a session-typed version of the Apache Spark data analytics framework, allowing the encoding of Spark applications without code modification.

A key concern in this line lies on whether session types can operate as or with MOM and whether data-centric properties can be achieved, all without giving up decentralized state management, a key microservice design pattern. For completeness, future solutions must also account for performance (Section 5.3), observability (Section 5.4), and security (Section 5.5) challenges analyzed in this work.

In terms of *development practices*, it is common for microservice developers to implement custom-built mechanisms for coordination [115], through Sagas [77] or patterns such as BFF. The design space for such approaches is diverse and can be realized either via synchronous or asynchronous protocols and via orchestration (centralized coordinator node to intermediate messages across microservices) or choreography (decentralized, as each microservice carries on its part and generates a message or request for the next microservice in the chain).

In asynchronous systems, where the unit of communication between components is abstracted away via well-defined event types, such as event-driven microservice architectures, the problems of composing multiple components, either via orchestration or choreography, are orthogonal. For example, having a central coordinator node mediating asynchronous messages across microservices through a message broker leads to, at least, the same, or in the worst case, extra challenges due to the additional indirection created. For instance, in the orchestration scheme, apart of microservices, the coordinator is also responsible to process events safely (CAS1), deal with weak delivery semantics (CAS7), coordinate rollbacking of states (CAS2), not to mention the need to evolve event schemas (CEM2) and the potential bottleneck in the figure of the coordinator (CP1 and CP3), which can impact the progress of steps across microservices. Therefore, the challenges and potential solutions described in this work apply to both cases.

Distributed event-based systems like *stream processing* are designed to perform continuous queries over unbounded data streams [28]. Computations in stream processing are expressed

through a dataflow, that is, a direct acyclic graph composed of operators. Each operator specifies a step of the declared query, such as a filter or a join. Input events flow through the operators, which in turn generate output events based on their computation for downstream processing. Microservice components can, in principle, be implemented as operators [186]. However, this has not arisen as a popular practice. By relying on static dataflow graphs, the stream processing computational model contrasts with pursued microservice principles such as loose-coupleness, autonomy, and dynamicity.

Complex Event Processing (CEP) targets pattern matching over event streams. While CEP can, in principle, be applied to challenges related to event processing order, like CAS3, it is unclear why developers favor their own application-level mechanisms to deal with these. From all mainstream message brokers, only Kafka provides CEP constructs through its KafkaStreams library, which can partially explain why CEP has not arisen as a popular practice in microservice architectures. While CEP can potentially support instances of CAS3 and CAS4, it is worth noting that CEP alone cannot fulfill many other challenges collected in this work, such as CAS1, CAS2, CAS5, CAS6, and CAS7. This leads microservice developers to be in charge of correctly interconnecting the CEP library with the microservices and MOM, adding additional complexity.

Summary. Table 8 exhibits a summary of the aforementioned discussed solutions. EDMA refers to event-driven microservice architecture. While all solutions share common limitations concerning the studied safety and liveness challenges, the popularity of EDMA may stem from the exploitation of high intra-and-inter component parallelism while being decoupled in time through a MOM like Kafka, allowing for highly configurable system, not to mention the support of industry-strength tools and technologies for state and message durability and recovery, evolution, and maintenance tasks.

7.2.2 Event Schema Management. Solutions based on languages and tools for defining message types and application programming interfaces for service-oriented computing have been historically devised [135, 157, 223]. We particularly highlight a recent stream of papers related to event schema management in microservice architectures.

According to Giallorenzo et al. [79], LEMMA [158] is a tool that applies **Model-Driven Engineering (MDE)** to provide a set of modeling languages and model processors to support the high-level specification and implementation of microservice domain models following DDD [61]. Other similar tools that apply MDE for microservices include MDSL [223], MicroBuilder [193], and JHipster [101].

The LEMMA tool [157] embraces many aspects of the microservice practice in relation to event management as analyzed in this work, including the use of patterns like domain events and MOM like Kafka to send events that are expected to form state views in other microservices (e.g., through the CQRS pattern). Modeling languages like LEMMA has the potential to serve as a useful tool to support that events modeled have their properties properly communicated across development teams and that events comply with expected microservice interfaces at design time. In terms of event schema evolution, LEMMA can be integrated with data serialization tools like Apache Avro and Protobuf, allowing for the management of the evolution of event types through models. After, a custom model processor can detect schema changes and generate code or alerts accordingly. The two models generated (the LEMMA and the specific data serialization technology) must be maintained coherently by development teams.

Jolie is a programming language focused on the development of service-oriented applications [135], including out-of-the-box features for service configuration and coordination. Unlike MDE tools like LEMMA, Jolie's support for event schema evolution is code-centric, involving the explicit

Table 8. Summary of Solution Proposals in the Safety and Liveness Landscape

Solution	Application Abstraction	Concurrency Model	Communication	State Management and Durability	Correctness Guarantees
EDMA	Small, independently deployable software components, each responsible for a specific application functionality; Components are deprecated and introduced dynamically	Intra-component parallelism achieved via (parallel) event consumption; Inter-component parallelism achieved via queuing asynchronous operations	Asynchronous events, often adhering to a schema, via MOM; No target identification; Changes in the application topology do not affect the communication system nor existing components	Each component encapsulates a private state; Decentralized data management, no global synchronization on state updates; State durability provided by a DBMS; Message durability provided by MOM	Developers must coordinate offset management (or queue consumption acknowledgment) with private state updates; Correctness of state updates often delegated to underlying database; No global properties (e.g., deadlock freedom)
Choreographic programming	Global interaction protocol (i.e., choreography) prescribing the communication structure of a concurrent system; Once compiled, generate code projected from the choreography	Concurrency modeled as parallel execution of choreography instances (e.g., components) following the choreography, only constrained by the allowed message exchange order	Message-based, following the choreography specification; Topology changes require recompilation or adaptation of choreography	Application-defined, as in EDMA	Deadlock-free programs and type safety by construction; Require application-level synchronization for shared state accesses
Session types	Distributed and concurrent components adhering to predefined types for communication	Concurrency achieved via multiple sessions executing in parallel	Message-based, governed by session protocols; Supports synchronous or asynchronous message passing depending on implementation	External to the session type system, therefore, application-defined, as in EDMA	Protocol compliance, deadlock-freedom, and communication safety by construction; Require application-level synchronization or framework support for shared state accesses
Coordination programming	Extensible program (at library or language level) that decouples coordination from computation	Based on shared-space concurrency semantics	Communication occurs via operations on a shared space (e.g., tuple spaces)	Shared state represented as tuples in a coordination space; Durability depends on underlying implementation	Atomicity of tuple operations often guaranteed but no transactional semantics; Deadlock handling often at application level
Actor systems	Actor, a concurrent, independent unit of computation that can model a shared resource with changing local state	An actor often resembles a sequential process; Concurrency is achieved via pipelining and dynamic creation of actors	Asynchronous messages, enabled by a mail system with eventual delivery guarantees; Target identification is necessary	Encapsulated local states, often changed as part of message processing; Durability depends on underlying implementation	Depends on message ordering and delivery guarantees; Mutual-exclusion free by design; Divergence and deadlocks are possible, must be dealt at application or system level

encoding of the different event type versions, the code logic to handle different event type versions, and the update of the service composition configuration to properly handle the different versions.

It is worthy noting that programming languages and MDE tools alone cannot guarantee important correctness properties uncovered in our study such as delivery (Section 5.1.7) and event processing (Section 5.1.4) semantics, rollback of state changes (Section 5.1.2), as well as correctness criteria about the queried data (Section 5.1.6). On the other hand, although not part of LEMMA core features, LEMMA can be extended to allow expressing the expected event delivery and event processing semantics. This can inform better implementers about the expected event processing guarantees for different program logic. Giallorenzo et al. [79] integrate LEMMA with Jolie, paving the way for inheriting LEMMA features in Jolie. An open question is whether it is possible to convey solutions that support evolving schemas transactionally, that is, all microservices converge to the new event schema or none.

7.2.3 Observability. In relation to observability challenges, we highlight some research work in session types for communication channel monitoring. Jia et al. [102] build on Dimoulas et al. [58] work in correct blame assignment in contract systems, modeling processes that communicate asynchronously over channels by using message queues, similar to what is found in event-driven microservice architectures. The system proposed [102] can detect violation of the session type (e.g., an out-of-order message is sent or received), assigning blame to the party responsible for the violation, which can potentially aid the end-to-end understanding mentioned as a challenge by developers in Section 5.4.1.

However, the focus on behavioral contracts at the level of the communication structure prevents reasoning about data properties that are often expressed in the event payload [85]. In this sense, we note that dependent session types allow expressing properties about the data in the communicating messages [195]. For example, specifying that a message type can only be sent after a specific value has been received. This requirement can appear in many challenges uncovered in this work, like CAS2, CAS3, CAS5, CAS6, and so on. On the other hand, compared to session types, dependent session types appear to lag behind in terms of tooling maturity and support for industry-strength languages.

Furthermore, interruptions in the communication are also an important requirement in real-world event-driven microservices, as observed along the Safety and Liveness challenges (Section 5.1). In this regard, Hu et al. [88] extend Scribble, a protocol language based on multiparty session types [46], to support the specification of asynchronously interruptible conversations. The solution relies on introducing runtime monitors in participants (i.e., components) to check the conformance of each to the multiparty session types properties that are defined through a Python API, allowing for dynamic verification of distributed processes.

Another line of work proposes verifying data-aware processes [27]. Business processes can represent web services, such as microservices and orchestration. Having a holistic view of data and processes in a unified manner allows the assertion about not only the control flow of business processes but also the state and evolution of data. Recent work includes symbolic reasoning for verifying data-aware processes [80].

First-order monitors can detect complex temporal patterns in event streams that carry application data [16]. Traditional monitors rely on the assumption that the entire application state is a single set and updates come in order, which hardly applies to microservice architectures. Monitoring tools that support out-of-order arrival of individual events like TimelyMon [165] hold potential to better fit the event-processing model in microservice architectures. An open problem is the declarative aspect, on which developers are required to express application semantics through languages such

as metric first-order temporal logic [16] that differ from industry-established practices like the use of SQL constructs.

Session-type properties, data-aware processes, and monitors appear as promising mechanisms for addressing observability in distributed systems. However, based on the challenges uncovered in this work, we highlight some points of attention on tackling observability in event-driven microservice architectures. One relates to the complete monitoring of channels of communication between components [58], including the seamless integration of the MOM technology used by the microservice architecture without performance overhead and functionality restrictions (Section 5.3.3). The intrusiveness of runtime monitors relying on session-type properties can pose challenges to technology, performance, and security constraints found in real-world deployments. Besides, the performance overhead and degradation in the communication channel can lead to arbitrary behavior of microservices [90] (Section 5.4.1), data invariants that cannot be expressed via the events flowing in the communication channel (Section 4.3), and the gamut of programming languages, paradigms, frameworks, and data models to comply with form other classes of challenges for adoption.

8 Concluding Remarks

In this article, we mine and analyze several relevant SO questions to characterize the state of the practices and the challenges microservice developers face while managing events. We identify key patterns developers use while attempting to realize their functional and NFRs, suggesting a tension between achieving requirements related to data consistency, loose coupling, and performance.

To further understand these tensions, we manually examined 628 sampled questions and identified key challenges microservice developers face in varied application scenarios. These include issues related to queuing, delivering, and processing events at the application level, as well as monitoring and securing the event flow. Based on the myriad of practical findings, we provide actionable implications for messaging systems, framework maintainers, cloud providers, and researchers. The insights obtained from our work can benefit different stakeholders involved in the development and management of event-driven microservice architectures:

— *For practitioners*: The challenges reinforce the importance of reliable sources of information and the need for careful analysis of the documentation of the technologies they adopt. Practitioners should question gray literature with general and high-level instructions that often do not embrace the complex nature of problems faced in production settings. Practitioners should not overlook problem-prevention measures, which include running, testing, and deploying their applications under the expected workload prior to production. Besides, developers must be cautious about "reinventing-the-wheel" approaches, since for every possible design attempt, there are reports of previous attempts.

— *For researchers*: Microservice developers, on opting for asynchronous designs, end up encountering a myriad of challenges that reveal a surprising number of unmet needs. The varied categories of problems uncovered, together with the correctness properties looked for by developers, highlight timely requirements and provide a reference for future research, which should cut across and thus involve different research communities. In particular, the findings evidenced in this work should encourage research communities to investigate better systems, algorithms, tools, programming models, and improved support for this growing community of application developers.

— *For tool builders*: Documentation must explicitly highlight and address the problems faced in practice. They must also provide actions that can be taken to mitigate the shortcomings of the existing tools. Furthermore, vendors must be clear about what requirements and guarantees

can and cannot be properly achieved, as well as the functionality and performance tradeoffs of the tool.

— *For educators*: We advise incorporating distributed systems concepts in software engineering courses, in particular algorithms and systems guarantees, and their relationship with modern application development in the cloud. Yet, training students on how event management relates and makes its presence in modern application architectures, such as serverless, microservices, SaaS, to name some.

We hope that the results drive the reflection of microservice developers and researchers to escape from the dangers of asynchronous, event-based designs, and to foment the design of event management technologies that meet the expectations of microservice developers, respectively.

Lastly, given the ubiquity of EDAs in the modern cloud development landscape, we consider that the study of event management in other emerging cloud programming paradigms, such as serverless architectures, including stateless and stateful functions, moduliths, and webassembly applications, entails interesting and natural extensions of this work, both from a characterization and a comparison perspective.

Acknowledgments

We thank Jean Mello and Marcos Antonio Vaz Salles for early discussions and initial drafting of this manuscript. We also thank SO for making the dataset available and the reviewers for their suggestions to improve the work. Part of the computation done for this project was performed on the UCloud interactive HPC system.

References

[1] Yalemisew Abgaz, Andrew McCarren, Peter Elger, David Solan, Neil Lapuz, Marin Bivol, Glenn Jackson, Murat Yilmaz, Jim Buckley, and Paul Clarke. 2023. Decomposition of monolith applications into microservices architectures: A systematic review. *IEEE Transactions on Software Engineering* 49, 8 (2023), 4213–4242. DOI: https://doi.org/10.1109/TSE.2023.3287297

[2] Gul Agha. 1986. *Actors: A Model of Concurrent Computation in Distributed Systems*. The MIT Press. DOI: https://doi.org/10.7551/mitpress/1086.001.0001

[3] Tyler Akidau, Robert Bradshaw, Craig Chambers, Slava Chernyak, Rafael J. Fernández-Moctezuma, Reuven Lax, Sam McVeety, Daniel Mills, Frances Perry, Eric Schmidt, et al. 2015. The dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. *Proceedings of the VLDB Endowment* 8 (2015), 1792–1803.

[4] annemartijn. 2024. How to Recover from Missed Integration or Notification Events in Event Driven Architecture? Retrieved July 29, 2024 from https://stackoverflow.com/questions/65425071/how-to-recover-from-missed-integration-or-notification-events-in-event-driven-ar

[5] .NET Application Architecture Reference Apps. 2025. eShop. Retrieved from https://github.com/dotnet/eShop

[6] Michael Armbrust, Armando Fox, Rean Griffith, Anthony D. Joseph, Randy Katz, Andy Konwinski, Gunho Lee, David Patterson, Ariel Rabkin, Ion Stoica, et al. 2010. A view of cloud computing. *Communications of the ACM* 53, 4 (Apr. 2010), 50–58. DOI: https://doi.org/10.1145/1721654.1721672

[7] Imran Arshad. 2024. Microservices Client Acknowledgement and Event Sourcing. Retrieved July 29, 2024 from https://stackoverflow.com/questions/54451013/microservices-client-acknowledgement-and-event-sourcing

[8] Asad Awadia. 2024. Istio Request Tracing for Vert.x Event Bus Messages. Retrieved July 29, 2024 from https://stackoverflow.com/questions/51123104/istio-request-tracing-for-vert-x-event-bus-messages

[9] AWS. 2024. Lambda. Retrieved November 27, 2024 from https://aws.amazon.com/lambda/

[10] AWS. 2025. Amazon SQS Increases Maximum Message Payload Size to 1 MiB. Retrieved August 5, 2025 from https://aws.amazon.com/about-aws/whats-new/2025/08/amazon-sqs-max-payload-size-1mib/

[11] Microsoft Azure. 2025. Confidential Computing. Retrieved July 15, 2025 from https://learn.microsoft.com/pt-pt/azure/confidential-computing/confidential-nodes-aks-overview

[12] Mehdi Bagherzadeh, Nicholas Fireman, Anas Shawesh, and Raffi Khatchadourian. 2020. Actor concurrency bugs: A comprehensive study on symptoms, root causes, API usages, and differences. *Proceedings of the ACM on Programming Languages* 4, OOPSLA, Article 214 (Nov. 2020), 32 pages. DOI: https://doi.org/10.1145/3428282

[13] Mehdi Bagherzadeh and Raffi Khatchadourian. 2019. Going big: A large-scale study on what big data developers ask. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '19)*. ACM, New York, NY, 432–442. DOI: https://doi.org/10.1145/3338906.3338939

[14] Peter Bailis and Ali Ghodsi. 2013. Eventual consistency today: Limitations, extensions, and beyond. *Communications of the ACM* 56, 5 (May 2013), 55–63. DOI: https://doi.org/10.1145/2447976.2447992

[15] Alan Bandeira, Carlos Alberto Medeiros, Matheus Paixao, and Paulo Henrique Maia. 2019. We need to talk about microservices: An analysis from the discussions on StackOverflow. In *Proceedings of the 2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. IEEE, 255–259. DOI: https://doi.org/10.1109/MSR.2019.00051

[16] David Basin, Felix Klaedtke, Samuel Müller, and Eugen Zălinescu. 2015. Monitoring metric first-order temporal properties. *Journal of the ACM* 62, 2, Article 15 (May 2015), 45 pages. DOI: https://doi.org/10.1145/2699444

[17] Dave Berardi. 2025. Claim-Check Pattern: When to Split a Large Message into a Claim-Check and a Payload. Retrieved August 15, 2025 from https://akfpartners.com/growth-blog/claim-check-pattern

[18] Tim Berglund. 2024. Introduction to Apache Kafka Partitions. Retrieved June 8, 2024 from https://developer.confluent.io/courses/apache-kafka/partitions/

[19] Ivan Beschastnikh, Perry Liu, Albert Xing, Patty Wang, Yuriy Brun, and Michael D. Ernst. 2020. Visualizing distributed system executions. *ACM Transactions on Software Engineering and Methodology* 29, 2, Article 9 (Mar. 2020), 38 pages. DOI: https://doi.org/10.1145/3375633

[20] Kadeck Blog. 2025. Kafka Topic Naming Conventions. Retrieved July 15, 2025 from https://www.datastreamhouse.com/blog/kafka-topic-naming-conventions-5-recommendations-with-examples

[21] Michal Borowiecki. 2024. Data Consistency Across Microservices. Retrieved July 29, 2024 from https://stackoverflow.com/questions/43950808/data-consistency-across-microservices/44748028#44748028

[22] Alberto Brandolini. 2013. Introducing Event Storming. Retrieved July 14, 2025 from https://ziobrando.blogspot.com/2013/11/introducing-event-storming.html

[23] Susanne Braun, Stefan Deßloch, Eberhard Wolff, Frank Elberzhager, and Andreas Jedlitschka. 2021. Tackling consistency-related design challenges of distributed data-intensive systems: An action research study. In *Proceedings of the 15th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM '21)*. ACM, New York, NY, Article 20, 11 pages. DOI: https://doi.org/10.1145/3475716.3475771

[24] Sergey Bykov, Alan Geller, Gabriel Kliot, Jim Larus, Ravi Pandya, and Jorgen Thelin. 2010. *Orleans: A Framework for Cloud Computing*. Technical Report MSRTR2010-159. Retrieved from https://www.microsoft.com/en-us/research/publication/orleans-a-framework-for-cloud-computing/

[25] Hebert Cabane and Kleinner Farias. 2024. On the impact of event-driven architecture on performance: An exploratory study. *Future Generation Computer Systems* 153 (2024), 52–69.

[26] CallMeTheBreeze. 2024. What Is the Best Way to Implement a Fast, Scalable Statistics Aggregation Architecture? Retrieved July 29, 2024 from https://stackoverflow.com/questions/30583466/what-is-the-best-way-to-implement-a-fast-scalable-statistics-aggregation-archit

[27] Diego Calvanese, Silvio Ghilardi, Alessandro Gianola, Marco Montali, and Andrey Rivkin. 2019. Verification of data-aware processes: Challenges and opportunities for automated reasoning. In *Proceedings of the 2nd International Workshop on Automated Reasoning: Challenges, Applications, Directions, Exemplary Achievements (ARCADE '19)*. EPTCS, Vol. 311, 53–61. DOI: https://doi.org/10.4204/EPTCS.311.9

[28] Paris Carbone, Stephan Ewen, Gyula Fóra, Seif Haridi, Stefan Richter, and Kostas Tzoumas. 2017. State management in Apache Flink®: Consistent stateful distributed stream processing. *Proceedings of the VLDB Endowment* 10, 12 (2017), 1718–1729.

[29] Nicholas Carriero and David Gelernter. 1989. Linda in context. *Communications of the ACM* 32, 4 (Apr. 1989), 444–458. DOI: https://doi.org/10.1145/63334.63337

[30] John Chang. 2024. Can I Rely on ConnectionId for Security with API Gateway Websockets? Retrieved July 29, 2024 from https://stackoverflow.com/questions/61737655/can-i-rely-on-connectionid-for-security-with-api-gateway-websockets

[31] Chunyang Chen and Zhenchang Xing. 2016. Mining technology landscape from stack overflow. In *Proceedings of the 10th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, 1–10.

[32] Dmitry Chornyi. 2021. Engineering Uber's Next-Gen Payments Platform. Uber Technologies Inc. Retrieved March 8, 2021 from https://eng.uber.com/payments-platform

[33] Christian Paesante Chris. 2024. CQRS—Out of Order Messages. Retrieved July 29, 2024 from https://stackoverflow.com/questions/53270770/cqrs-out-of-order-messages

[34] Christian. 2024. Event Sourcing—Event Streams Clarification. Retrieved July 29, 2024 from https://stackoverflow.com/questions/53949531/event-sourcing-event-streams-clarification

[35] Christophe. 2024. Is It Bad Practice to Produce and Consume Messages from the Same Topic? Retrieved July 29, 2024 from https://stackoverflow.com/questions/45486658/is-it-bad-practice-to-produce-and-consume-messages-from-the-same-topic
[36] L. Chung, B. A. Nixon, E. Yu, and J. Mylopoulos. 2012. *Non-Functional Requirements in Software Engineering*. Springer, New York, NY. Retrieved from https://books.google.dk/books?id=MNrcBwAAQBAJ
[37] Google Cloud. 2024. What Is a Hybrid Cloud? Retrieved June 19, 2024 from https://cloud.google.com/learn/what-is-hybrid-cloud
[38] E. F. Codd. 1970. A relational model of data for large shared data banks. *Communications of the ACM* 13, 6 (June 1970), 377–387. DOI: https://doi.org/10.1145/362384.362685
[39] Codependent. 2024. How to Implement a Microservice Event Driven Architecture with Spring Cloud Stream Kafka and Database Per Service. Retrieved July 29, 2024 from https://stackoverflow.com/questions/42140285/how-to-implement-a-microservice-event-driven-architecture-with-spring-cloud-stre
[40] Codependent. 2024. Spring Cloud Stream Kafka—Eventual Consistency—Does Kafka Auto Retry Unacknowledged Messages (When Using autocommitoffset=false). Retrieved June 19, 2024 from https://stackoverflow.com/questions/42230797/spring-cloud-stream-kafka-eventual-consistency-does-kafka-auto-retry-unackno
[41] Confluent. 2025. Kafka Consumer for Confluent Platform. Retrieved July 14, 2025 from https://docs.confluent.io/platform/current/clients/consumer.html
[42] Confluent, Inc. 2023. Schema Registry Overview. Confluent, Inc. Retrieved July 2023 from https://docs.confluent.io/platform/current/schema-registry/index.html
[43] RxJava Contributors. 2018. Backpressure (2.0)—ReactiveX/RxJava Wiki. Retrieved September 09, 2025 from https://github.com/ReactiveX/RxJava/wiki/Backpressure-(2.0)
[44] Rxjs Contributors. 2024. ReactiveX/rxjs: A Reactive Programming Library for JavaScript. Retrieved September 09, 2025 from https://github.com/ReactiveX/rxjs/
[45] RSocket Contributors. 2024. RSocket—Application Protocol Providing Reactive Streams Semantics. Retrieved September 09, 2025 from https://rsocket.io/
[46] Mario Coppo, Mariangiola Dezani-Ciancaglini, Luca Padovani, and Nobuko Yoshida. 2015. *A Gentle Introduction to Multiparty Asynchronous Session Types*. Springer International Publishing, Cham, 146–178. DOI: https://doi.org/10.1007/978-3-319-18941-3_4
[47] Microsoft Corporation. 2023. Durable Task Framework. Retrieved September 11, 2025 from https://github.com/Azure/durabletask
[48] Ana Crudu and MoldStud Research Team. 2024. Understanding Event-Driven Architecture—Key to Scalable Software Systems. Retrieved August 10, 2025 from https://moldstud.com/articles/p-understanding-the-role-of-event-driven-architecture-in-scalable-software-systems
[49] Werner Damm and David Harel. 2001. From MSCs to Statecharts. In *Formal Methods for Embedded Distributed Systems*. Jørgen Staunstrup (Ed.), Springer, 77–105. DOI: https://doi.org/10.1007/978-0-387-35570-2_5
[50] Werner Damm and David Harel. 2001. LSCs: Breathing life into message sequence charts. *Formal Methods in System Design* 19, 1 (2001), 45–80. DOI: https://doi.org/10.1023/A:1011227529550
[51] Dapr. 2024. Workflow Patterns. Retrieved July 10, 2024 from https://docs.dapr.io/developing-applications/building-blocks/workflow/workflow-patterns
[52] Dapr. 2024. Distributed Application Runtime. Retrieved October16, 2024 from https://dapr.io/
[53] Datadog. 2023. Datadog—Modern Monitoring & Security. Retrieved July 2023 from https://www.datadoghq.com/
[54] Deblearns1. 2024. Kafka Implementation—How to See End to End Workflows with Microservices. Retrieved July 29, 2024 from https://stackoverflow.com/questions/65500926/kafka-implementation-how-to-see-end-to-end-workflows-with-microservices
[55] Shalaka Deshpande. 2024. How to Restrict Publishing to a Topic in GCP Pub/Sub Based on Number of Unacknowledged Messages in the Queue? Retrieved July 29, 2024 from https://stackoverflow.com/questions/74064658/how-to-restrict-publishing-to-a-topic-in-gcp-pub-sub-based-on-number-of-unacknow
[56] Santanu Dey. 2024. Data Consistency Across Microservices. Retrieved July 29, 2024 from https://stackoverflow.com/questions/43950808/data-consistency-across-microservices
[57] Paolo Di Francesco, Patricia Lago, and Ivano Malavolta. 2018. Migrating towards microservice architectures: An industrial survey. In *Proceedings of the 2018 IEEE International Conference on Software Architecture (ICSA)*. IEEE, 29–2909. DOI: https://doi.org/10.1109/ICSA.2018.00012
[58] Christos Dimoulas, Sam Tobin-Hochstadt, and Matthias Felleisen. 2012. Complete monitors for behavioral contracts. In *Programming Languages and Systems*. Helmut Seidl (Ed.), Springer, Berlin, 214–233.
[59] Uber Engineering. 2020. Revolutionizing Money Movements at Scale with Strong Data Consistency. Uber Technologies Inc. Retrieved March 8, 2021 from https://eng.uber.com/money-scale-strong-data
[60] European Union (EU). 2016. General Data Protection Regulation (GDPR). Retrieved July 28, 2025 from https://gdpr.eu/

[61] Eric Evans. 2004. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional.

[62] EventCatalog. 2025. Bring Discoverability to Your Architecture. Retrieved July 14, 2025 from https://eventcatalog.dev

[63] Kleinner Farias and Luan Lazzari. 2023. Event-driven architecture and REST architectural style: An exploratory study on modularity. *Journal of Applied Research and Technology* 21, 3 (June 2023), 338–351. DOI: https://doi.org/10.22201/icat.24486736e.2023.21.3.1764

[64] Danica Fine and Nikoleta Verbeck. 2022. Diagnose and Debug Apache Kafka Issues: Understanding Increased Consumer Rebalance Time: Increased Consumer Rebalance Time. Retrieved June 19, 2024 from https://www.confluent.io/blog/debug-apache-kafka-pt-3/

[65] Apache Foundation. 2025. Avro. Retrieved July 14, 2025 from https://avro.apache.org

[66] Apache Software Foundation. 2024. Apache Flink—Stateful Computations Over Data Streams. Retrieved September 09, 2025 from https://flink.apache.org/

[67] OpenJS Foundation. 2025. Node.js—Backpressuring in Streams. Retrieved September 09, 2025 from https://nodejs.org/en/learn/modules/backpressuring-in-streams

[68] Martin Fowler. 2005. Event Sourcing: Capture All Changes to an Application State as a Sequence of Events. Martin Fowler. Retrieved May 22, 2025 from https://martinfowler.com/eaaDev/EventSourcing.html

[69] Martin Fowler. 2006. Focusing on Events. Martin Fowler. Retrieved July 3, 2024 from https://martinfowler.com/eaaDev/EventNarrative.html

[70] Martin Fowler. 2011. CQRS. Retrieved May 22, 2025 from https://martinfowler.com/bliki/CQRS.html

[71] Martin Fowler. 2014. Microservices a Definition of This New Architectural Term. Martin Fowler. Retrieved July 2023 from https://martinfowler.com/articles/microservices.html

[72] Martin Fowler. 2017. What Do You Mean by "Event-Driven"? Martin Fowler. Retrieved June 22, 2024 from https://martinfowler.com/articles/201701-event-driven.html

[73] Marios Fragkoulis, Paris Carbone, Vasiliki Kalavri, and Asterios Katsifodimos. 2023. A survey on the evolution of stream processing systems. *The VLDB Journal* 33, 2 (Mar. 2024), 507–541. DOI: https://doi.org/10.1007/s00778-023-00819-8

[74] Paolo Francesco, Ivano Malavolta, and Patricia Lago. 2017. Research on architecting microservices: Trends, focus, and potential for industrial adoption. In *Proceedings of the International Conference on Software Architecture*, 21–30. DOI: https://doi.org/10.1109/ICSA.2017.24

[75] Victor França. 2024. Saga Choreography Implementation Problems. Retrieved July 29, 2024 from https://stackoverflow.com/questions/48487098/saga-choreography-implementation-problems

[76] Renato Gama. 2024. How to Trigger an Action Only after Receiving Two or More Events in an Event Driven Architecture? Retrieved July 29, 2024 from https://stackoverflow.com/questions/51421205/how-to-trigger-an-action-only-after-receiving-two-or-more-events-in-an-event-dri

[77] Hector Garcia-Molina and Kenneth Salem. 1987. Sagas. In *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data (SIGMOD '87)*. ACM, New York, NY, 249–259. DOI: https://doi.org/10.1145/38713.38742

[78] David Gelernter and Nicholas Carriero. 1992. Coordination languages and their significance. *Communications of the ACM* 35, 2 (Feb. 1992), 97–107. DOI: https://doi.org/10.1145/129630.129635

[79] Saverio Giallorenzo, Fabrizio Montesi, Marco Peressotti, and Florian Rademacher. 2023. LEMMA2Jolie: A tool to generate microservice APIs from domain models. *Science of Computer Programming* 228 (2023), 102956. DOI: https://doi.org/10.1016/j.scico.2023.102956

[80] Alessandro Gianola. 2023. *Verification of Data-Aware Processes via Satisfiability Modulo Theories*. Lecture Notes in Business Information Processing, Vol. 470, Springer, Cham. DOI: https://doi.org/10.1007/978-3-031-42746-6

[81] Adam Gluck. 2024. *Introducing Domain-Oriented Microservice Architecture*. Uber Technologies Inc. Retrieved from https://www.uber.com/en-DK/blog/microservice-architecture/

[82] Nicolas E. Gold and Jens Krinke. 2022. Ethics in the mining of software repositories. *Empirical Software Engineering* 27, 1 (2022), 17.

[83] Dimitrios Gravanis, George Kakarontzas, and Vassilis Gerogiannis. 2022. You don't need a microservices architecture (yet): Monoliths may do the trick. In *Proceedings of the 2021 European Symposium on Software Engineering (ESSE '21)*. ACM, New York, NY, 39–44. DOI: https://doi.org/10.1145/3501774.3501780

[84] Tuna Hacaloglu and Onur Demirors. 2023. An exploratory case study using events as a software size measure. *Information Technology and Management* 24 (Apr. 2023), 1–20. DOI: https://doi.org/10.1007/s10799-023-00394-y

[85] Klaus Havelund, Giles Reger, Daniel Thoma, and Eugen Zălinescu. 2018. *Monitoring Events That Carry Data*. Springer International Publishing, Cham, 61–102. DOI: https://doi.org/10.1007/978-3-319-75632-5_3

[86] hendoe. 2024. Approaches to Update Microservices Databases Retroactively. Retrieved July 29, 2024 from https://stackoverflow.com/questions/59923240/approaches-to-update-microservices-databases-retroactively

[87] hsingh. 2025. How to Measure the Progress of an Event Processing in Case of the Event Is Going through Multiple Micro-Services? Retrieved May 5, 2025 from https://stackoverflow.com/questions/73397625/how-to-measure-the-progress-of-an-event-processing-in-case-of-the-event-is-going

[88] Raymond Hu, Rumyana Neykova, Nobuko Yoshida, Romain Demangeon, and Kohei Honda. 2013. Practical interruptible conversations. In *Runtime Verification*. Axel Legay and Saddek Bensalem (Eds.), Springer, Berlin, 130–148.

[89] Edward Huang. 2020. How to Determine Ordering in Microservices? Retrieved July 2023 from https://edward-huang.com/cloud/programming/tech/distributed-system/architecture/2020/12/21/how-to-determine-ordering-in-microservices/

[90] Lexiang Huang, Matthew Magnusson, Abishek Bangalore Muralikrishna, Salman Estyak, Rebecca Isaacs, Abutalib Aghayev, Timothy Zhu, and Aleksey Charapko. 2022. Metastable failures in the wild. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22)*. USENIX Association, 73–90. Retrieved from https://www.usenix.org/conference/osdi22/presentation/huang-lexiang

[91] Fabian Hueske and Vasiliki Kalavri. 2019. *Stream Processing with Apache Flink: Fundamentals, Implementation, and Operation of Streaming Applications*. O'Reilly Media.

[92] Syed Hussain, Muhammad Khan, Ehab Al-Shaer, and Fatih Aysan. 2021. SoK: Function-as-a-Service: From an application developer's perspective. *Journal of Systems Research* 1, 1 (2021), 1–20.

[93] Yahya Hussein. 2025. Messaging Between Services Events vs Commands. Retrieved November 1, 2025 from https://stackoverflow.com/questions/47427629/messaging-between-services-events-vs-commands

[94] Hans Hüttel, Ivan Lanese, Vasco T. Vasconcelos, Luís Caires, Marco Carbone, Pierre-Malo Deniélou, Dimitris Mostrous, Luca Padovani, António Ravara, Emilio Tuosto, et al. 2016. Foundations of session types and behavioural contracts. *ACM Computing Surveys* 49, 1, Article 3 (Apr. 2016), 36 pages. DOI: https://doi.org/10.1145/2873052

[95] Roberto Ierusalimschy, Luiz Henrique de Figueiredo, and Waldemar Celes Filho. 1996. Lua—An extensible extension language. *Software: Practice and Experience* 26, 6 (June 1996), 635–652.

[96] Internet Engineering Task Force (IETF). 2021. QUIC: A UDP-Based Multiplexed and Secure Transport. Retrieved August 5, 2025 from https://datatracker.ietf.org/doc/rfc9000/

[97] ignacy. 2024. Starting New Kafka Streams Microservice, When There Is Data Retention Period on Input Topics. Retrieved July 29, 2024 from https://stackoverflow.com/questions/67074772/starting-new-kafka-streams-microservice-when-there-is-data-retention-period-on

[98] NEW RELIC INC. 2023. Introduction to New Relic for Go. Retrieved July 2023 from https://docs.newrelic.com/docs/apm/agents/go-agent/get-started/introduction-new-relic-go/

[99] Workflow Patterns Initiative. 2024. Workflow Patterns. Retrieved July 10, 2024 from http://www.workflowpatterns.com

[100] Jean-Marie Jacquet, Isabelle Linden, and Mihail-Octavian Staicu. 2012. Blackboard rules for coordinating context-aware applications in mobile ad hoc networks. In *Proceedings of the 10th International Workshop on Foundations of Coordination Languages and Software Architectures (FOCLASA '12)*. EPTCS, Vol. 91, 63–78. DOI: https://doi.org/10.4204/EPTCS.91.5

[101] JHipster. 2024. JHipster Domain Language (JDL). Retrieved July 29, 2024 from https://www.jhipster.tech/jdl/intro/

[102] Limin Jia, Hannah Gommerstadt, and Frank Pfenning. 2016. Monitors and blame assignment for higher-order session types. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '16)*. ACM, New York, NY, 582–594. DOI: https://doi.org/10.1145/2837614.2837662

[103] Joe Gates. 2024. How to Deal with Replication Lag in Microservices. Retrieved July 29, 2024 from https://stackoverflow.com/questions/71971389/how-to-deal-with-replication-lag-in-microservices

[104] Jordi. 2024. Saga Pattern: What About If Compensation Action Fails. Retrieved July 29, 2024 from https://stackoverflow.com/questions/73108942/saga-pattern-what-about-if-compensation-action-fails

[105] jsc31994. 2024. Why Is Kafka Streams in Node Sending Me Duplicate Payloads After I Terminate the Process and Run Once More? Retrieved July 29, 2024 from https://stackoverflow.com/questions/63391504/why-is-kafka-streams-in-node-sending-me-duplicate-payloads-after-i-terminate-the

[106] jub0bs. 2025. Event Sourcing Microservices: How to Manage Timestamp. Retrieved August 7, 2025 from https://stackoverflow.com/questions/41082938/event-sourcing-microservices-how-to-manage-timestamp

[107] Julio. 2024. Microservices Async Communication Circuit Breaker. Retrieved July 29, 2024 from https://stackoverflow.com/questions/68307199/microservices-async-communication-circuit-breaker

[108] Apache Kafka. 2021. Kafka Streams. Retrieved May 2, 2021 from https://kafka.apache.org/27/documentation/streams

[109] KitKarson. 2024. Data Dependency Among Microservices. Retrieved July 29, 2024 from https://stackoverflow.com/questions/70509292/data-dependency-among-microservices

[110] Martin Kleppmann, Alastair R. Beresford, and Boerge Svingen. 2019. Online event processing: Achieving consistency where distributed transactions have failed. *Queue* 17, 1 (Feb. 2019), 116–136. DOI: https://doi.org/10.1145/3317287.3321612

[111] Jay Kreps, Neha Narkhede, and Jun Rao. 2011. Kafka: A distributed messaging system for log processing. In *Proceedings of the NetDB*, Vol. 11, 1–7.
[112] Grafana Labs. 2023. Grafana Labs—Operational Dashboards for Your Data Here, There, or Anywhere. Retrieved July 2023 from https://grafana.com/
[113] Nokia Bell Labs. 2000. Specification and Verification of Message Sequence Charts. Nokia. Retrieved September 16, 2024 from https://www.nokia.com/bell-labs/publications-and-media/publications/specification-and-verification-of-message-sequence-charts/
[114] Rodrigo Laigner, Marcos Kalinowski, Pedro Diniz, Leonardo Barros, Carlos Cassino, Melissa Lemos, Darlan Arruda, Sérgio Lifschitz, and Yongluan Zhou. 2020. From a monolithic big data system to a microservices event-driven architecture. In *Proceedings of the 2020 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. IEEE, 213–220. DOI: https://doi.org/10.1109/SEAA51224.2020.00045
[115] Rodrigo Laigner, Yongluan Zhou, Marcos Antonio Vaz Salles, Yijian Liu, and Marcos Kalinowski. 2021. Data management in microservices: State of the ractice, challenges, and research directions. *Proceedings of the VLDB Endowment* 14, 13 (2021), 3348–3361. DOI: https://doi.org/10.14778/3484224.3484232
[116] L. Lamport. 1977. Proving the correctness of multiprocess programs. *IEEE Transactions on Software Engineering* SE-3, 2 (1977), 125–143. DOI: https://doi.org/10.1109/TSE.1977.229904
[117] Lerman. 2024. Internal k8s Services Communication Not Balanced. Retrieved July 29, 2024 from https://stackoverflow.com/questions/73349618/internal-k8s-services-communication-not-balanced
[118] Anna Lesniak, Rodrigo Laigner, and Yongluan Zhou. 2021. Enforcing consistency in microservice architectures through event-based constraints. In *Proceedings of the International Conference on Distributed and Event-Based Systems (DEBS)*, 180–183.
[119] Qian Li, Peter Kraft, Michael Cafarella, Çağatay Demiralp, Goetz Graefe, Christos Kozyrakis, Michael Stonebraker, Lalith Suresh, Xiangyao Yu, and Matei Zaharia. 2023. R3: Record-replay-retroaction for database-backed applications. *Proceedings of the VLDB Endowment* 16, 11 (July 2023), 3085–3097. DOI: https://doi.org/10.14778/3611479.3611510
[120] Zijun Li, Linsong Guo, Jiagan Cheng, Quan Chen, Bingsheng He, and Minyi Guo. 2022. The serverless computing survey: A technical primer for design architecture. *ACM Computing Surveys* 54, 10s, Article 220 (Sept. 2022), 34 pages. DOI: https://doi.org/10.1145/3508360
[121] B. Liskov, D. Curtis, P. Johnson, and R. Scheifer. 1987. Implementation of argus. *ACM SIGOPS Operating Systems Review* 21, 5 (1987), 111–122. Retrieved from https://doi.org/10.1145/37499.37514
[122] Dynatrace LLC. 2023. Dynatrace—Application Observability. Retrieved July 2023 from https://www.dynatrace.com/platform/applications-microservices-monitoring/
[123] NServiceBus Ltd. 2025. NServiceBus—Particular Software. Retrieved September 09, 2025 from https://particular.net/nservicebus
[124] Florian Ludewig. 2024. Deleting Events with Tombstones and Public/Private Data in an Event-Sourced System. Retrieved July 29, 2024 from https://stackoverflow.com/questions/57784098/deleting-events-with-tombstones-and-public-private-data-in-an-event-sourced-sy
[125] M. Benjamin. 2024. CQRS + Microservices: How to Handle Relations/Validation? Retrieved July 29, 2024 from https://stackoverflow.com/questions/42528718/cqrs-microservices-how-to-handle-relations-validation
[126] Qusay H. Mamoud. 2005. Getting Started With JavaSpaces Technology: Beyond Conventional Distributed Programming Paradigms. Oracle. Retrieved September 16, 2024 from https://www.oracle.com/technical-resources/articles/javase/javaspaces.html
[127] Materialize. 2025. Kafka Streams. Retrieved September 09, 2025 from https://materialize.com/
[128] Matthias. 2024. Spring Security OAuth2 AuthorizationServer. Retrieved July 29, 2024 from https://stackoverflow.com/questions/29148547/spring-security-oauth2-authorizationserver
[129] Nabor C. Mendonça, Craig Box, Costin Manolache, and Louis Ryan. 2021. The monolith strikes back: Why Istio migrated from microservices to a monolithic architecture. *IEEE Software* 38, 5 (2021), 17–22.
[130] Hamdy Michael Ayas, Philipp Leitner, and Regina Hebig. 2023. An empirical study of the systemic and technical migration towards microservices. *Empirical Software Engineering* 28, 4 (2023), 85.
[131] Microsoft. 2025. Configure Customer-Managed Keys for Encrypting Azure Service Bus Data at Rest. Retrieved September 09, 2025 from https://docs.microsoft.com/en-us/azure/service-bus-messaging/configure-customer-managed-key
[132] C. Mohan and B. Lindsay. 1985. Efficient commit protocols for the tree of processes model of distributed transactions. *ACM SIGOPS Operating Systems Review* 19, 2 (Apr. 1985), 40–52. DOI: https://doi.org/10.1145/850770.850772
[133] Fabrizio Montesi. 2016. Kickstarting choreographic programming. In *Web Services, Formal Methods, and Behavioral Types*. Thomas Hildebrandt, António Ravara, Jan Martijn van der Werf, and Matthias Weidlich (Eds.), Springer International Publishing, Cham, 3–10.
[134] Fabrizio Montesi. 2023. *Introduction to Choreographies*. Cambridge University Press.

[135] Fabrizio Montesi, Claudio Guidi, and Gianluigi Zavattaro. 2014. *Service-Oriented Programming with Jolie*. Springer, New York, NY, 81–107. DOI: https://doi.org/10.1007/978-1-4614-7518-7_4

[136] mozilla.org Contributors. 2025. Streams API. Retrieved September 09, 2025 from https://developer.mozilla.org/en-US/docs/Web/API/Streams_API

[137] Gastón Márquez, Mónica M. Villegas, and Hernán Astudillo. 2018. An empirical study of scalability frameworks in open source microservices-based systems. In *Proceedings of the 2018 37th International Conference of the Chilean Computer Science Society (SCCC)*, 1–8. DOI: https://doi.org/10.1109/SCCC.2018.8705256

[138] Ali Rezaei Nasab, Mojtaba Shahin, Seyed Ali Hoseyni Raviz, Peng Liang, Amir Mashmool, and Valentina Lenarduzzi. 2023. An empirical study of security practices for microservices systems. *Journal of Systems and Software* 198 (2023), 111563.

[139] Arthur Navarro, Julien Ponge, Frédéric Le Mouël, and Clément Escoffier. 2023. Considerations for integrating virtual threads in a Java framework: A Quarkus example in a resource-constrained environment. In *Proceedings of the 17th ACM International Conference on Distributed and Event-Based Systems (DEBS '23)*. ACM. DOI: https://doi.org/10.1145/3583678.3596895

[140] Sam Newman. 2015. *Building Microservices* (1st ed.). O'Reilly Media, Inc., Sebastopol, CA.

[141] nik2o. 2024. Join Data in CQRS Patterns from Different Microservices. Retrieved July 29, 2024 from https://stackoverflow.com/questions/57328269/join-data-in-cqrs-patterns-from-different-microservices

[142] Nubank. 2017. Architecting a Modern Financial Institution. Nubank. Retrieved February 11, 2022 from https://www.infoq.com/presentations/nubank-architecture/

[143] Nubank. 2019. Microservices at Nubank, an Overview. Nubank. Retrieved March 8, 2021 from https://building.nubank.com.br/microservices-at-nubank-an-overview

[144] Odsh. 2024. CQRS: Project Out-of-Order Notifications in an ElasticSearch Read Model. Retrieved July 29, 2024 from https://stackoverflow.com/questions/47516458/cqrs-project-out-of-order-notifications-in-an-elasticsearch-read-model

[145] D. Ouardi, M. Barkallah, and J. Jacquet. 2024. The B2Scala tool: Integrating Bach in Scala with security in mind. *Electronic Proceedings in Theoretical Computer Science* 414 (2024), 58–76.

[146] Michiel Overeem, Marten Spoor, and Slinger Jansen. 2017. The dark side of event sourcing: Managing data conversion. In *Proceedings of the 2017 IEEE 24th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 193–204.

[147] Michiel Overeem, Marten Spoor, Slinger Jansen, and Sjaak Brinkkemper. 2021. An empirical characterization of event sourced systems and their schema evolution—Lessons from industry. *Journal of Systems and Software* 178 (2021), 110970. DOI: https://doi.org/10.1016/j.jss.2021.110970

[148] oznomal. 2024. How to Monitor REST Endpoint for Long Running Jobs. Retrieved July 29, 2024 from https://stackoverflow.com/questions/56923600/how-to-monitor-rest-endpoint-for-long-running-jobs

[149] Christian Paesante. 2024. AWS Event-Sourcing Implementation. Retrieved July 29, 2024 from https://stackoverflow.com/questions/52632129/aws-event-sourcing-implementation

[150] Igor Petrov. 2024. SAGA and Local Transactions with Kafka and Postgres in Spring Boot. Retrieved July 29, 2024 from https://stackoverflow.com/questions/62223553/saga-and-local-transactions-with-kafka-and-postgres-in-spring-boot

[151] PostgreSQL. 2025. Large Objects. Retrieved July 15, 2025 from https://www.postgresql.org/docs/16/largeobjects.html

[152] Praveen. 2024. Aggregates in Event Sourcing Pattern. Retrieved July 29, 2024 from https://stackoverflow.com/questions/49985156/aggregates-in-event-sourcing-pattern

[153] Roger Pressman. 2009. *Software Engineering: A Practitioner's Approach* (7th ed.). McGraw-Hill, Inc.

[154] Dan Pritchett. 2008. BASE: An acid alternative: In partitioned databases, trading some consistency for availability can lead to dramatic improvements in scalability. *Queue* 6, 3 (May 2008), 48–55. DOI: https://doi.org/10.1145/1394127.1394128

[155] Psyxto. 2024. Eventual Consistency Data Orchestration in Microservices RabbitMq Broker. Retrieved July 29, 2024 from https://stackoverflow.com/questions/71160328/eventual-consistency-data-orchestration-in-microservices-rabbitmq-broker

[156] Maciej Pszczolinski. 2024. DB Transaction and Integrations Events Dispatch—How to Make It Atomic? Retrieved July 29, 2024 from https://stackoverflow.com/questions/62364508/db-transaction-and-integrations-events-dispatch-how-to-make-it-atomic

[157] Florian Rademacher, Jonas Sorgalla, Philip Wizenty, and Simon Trebbau. 2021. Towards holistic modeling of microservice architectures using LEMMA. In *ECSA 2021 Companion Volume, Virtual*. Robert Heinrich, Raffaela Mirandola, and Danny Weyns (Eds.), *CEUR Workshop Proceedings*, Vol. 2978, CEUR-WS.org. Retrieved from https://ceur-ws.org/Vol-2978/mde4sa-paper2.pdf

[158] Florian Rademacher, Philip Wizenty, Jonas Sorgalla, Sabine Sachweh, and Albert Zündorf. 2024. *Model-Driven Engineering of Microservice Architectures—The LEMMA Approach*. Springer Nature, Cham, 105–147. DOI: https://doi.org/10.1007/978-3-031-44412-8_5

[159] raduone. 2024. Can the Consumer Slow Down the Producer, If the Producer Is in a Different Service, Using Reactor Kafka? Retrieved July 29, 2024 from https://stackoverflow.com/questions/57181251/can-the-consumer-slow-down-the-producer-if-the-producer-is-in-a-different-servi

[160] Milad Raeisi. 2024. Microservice Design: Change on Data Requirements or Schema. Retrieved July 29, 2024 from https://stackoverflow.com/questions/68781332/microservice-design-change-on-data-requirements-or-schema

[161] Rafiq. 2024. How to Handle Multiple Update Event When There Is More Then One Replica of a Pod. Retrieved July 29, 2024 from https://stackoverflow.com/questions/67612615/how-to-handle-multiple-update-event-when-there-is-more-then-one-replica-of-a-pod

[162] Francisco Ramírez, Carlos Mera-Gómez, Rami Bahsoon, and Yuqun Zhang. 2021. An empirical study on microservice software development. In *Proceedings of the 2021 IEEE/ACM Joint 9th International Workshop on Software Engineering for Systems-of-Systems and 15th Workshop on Distributed Software Development, Software Ecosystems and Systems-of-Systems (SESoS/WDES)*. IEEE, 16–23.

[163] Tharindu Ranasingha. 2024. Is It Wise to Stream File Over Vertx Event Bus. Retrieved July 29, 2024 from https://stackoverflow.com/questions/51665264/is-it-wise-to-stream-file-over-vertx-event-bus

[164] Rasmond. 2024. What to Do When Exception Is Thrown After State Is Modified? Retrieved July 29, 2024 from https://stackoverflow.com/questions/73928296/what-to-do-when-exception-is-thrown-after-state-is-modified

[165] Lennard Reese, Rafael Castro G. Silva, and Dmitriy Traytel. 2024. TimelyMon: A streaming parallel first-order monitor. In *Proceedings of the 24th International Conference on Runtime Verification (RV '24)*. Springer-Verlag, Berlin, 150–160. DOI: https://doi.org/10.1007/978-3-031-74234-7_9

[166] N. De La Rocque Rodriguez, R. Almeida De Miranda Santos, and R. Ierusalimschy. 2021. Events in the MPA System. In *Monografias em Ciencia da Computacao*. PUC-Rio, Article 1, 11 pages. DOI: https://doi.org/10.17771/PUCRio.DImcc.60952

[167] Chris Richardson. 2023. A Pattern Language for Microservices. Retrieved October 1, 2023 from https://microservices.io/patterns/index.html

[168] Chris Richardson. 2022. Chris Richardson Consulting, Inc. Retrieved March 2, 2022 from https://microservices.io/patterns/data/domain-event.html

[169] Robert. 2024. Event Publisher for ASP.NET Web API. Retrieved July 29, 2024 from https://stackoverflow.com/questions/39305118/event-publisher-for-asp-net-web-api

[170] sam, 2024. Refreshing Microservice Data with Event Messages. Retrieved July 29, 2024 from https://stackoverflow.com/questions/64049385/refreshing-microservice-data-with-event-messages

[171] Santosh. 2024. Best Way to Track/Trace a JSON Object (a Time Series Data) as It Flows Through a System of Microservices on a IOT Platform. Retrieved July 29, 2024 from https://stackoverflow.com/questions/63699141/best-way-to-track-trace-a-json-object-a-time-series-data-as-it-flows-through-a

[172] Nitin Sarma. 2020. Event Driven Architectures @ Netflix Content Finance Engineering. Netflix. Retrieved April 8, 2024 from https://www.linkedin.com/pulse/event-driven-architectures-netflix-content-finance-engineering-s/

[173] Alceste Scalas, Nobuko Yoshida, and Elias Benussi. 2019. Effpi: Verified message-passing programs in Dotty. In *Proceedings of the 10th ACM SIGPLAN Symposium on Scala (Scala '19)*. ACM, New York, NY, 27–31. DOI: https://doi.org/10.1145/3337932.3338812

[174] Georg Schwarz. 2025. Microservices Async Communication Circuit Breaker. Retrieved July 28, 2025 from https://stackoverflow.com/questions/67582691/scaling-kafka-for-microservices

[175] Airbnb Engineering & Data Science. 2018. Measuring Transactional Integrity in Airbnb's Distributed Payment Ecosystem. Retrieved February 25, 2023 from https://medium.com/airbnb-engineering/measuring-transactional-integrity-in-airbnbs-distributed-payment-ecosystem-a670d6926d22

[176] Airbnb Engineering & Data Science. 2019. Avoiding Double Payments in a Distributed Payments System. Retrieved March 8, 2021 from https://medium.com/airbnb-engineering/avoiding-double-payments-in-a-distributed-payments-system-2981f6b070bb

[177] C. B. Seaman. 1999. Qualitative methods in empirical studies of software engineering. *IEEE Transactions on Software Engineering* 25, 4 (1999), 557–572. DOI: https://doi.org/10.1109/32.799955

[178] Sesigl. 2024. Use Kafka to Increase Resilience of a New Service. Retrieved July 29, 2024 from https://stackoverflow.com/questions/48912603/use-kafka-to-increase-resilience-of-a-new-service

[179] Natan Silnitsky. 2022. Event Driven Architecture—5 Pitfalls to Avoid. Wix. Retrieved July 1, 2023 from https://medium.com/wix-engineering/event-driven-architecture-5-pitfalls-to-avoid-b3ebf885bdb1

[180] Natan Silnitsky. 2022. Troubleshooting Kafka for 2000 Microservices at Wix. Wix. Retrieved July 1, 2023 from https://medium.com/wix-engineering/troubleshooting-kafka-for-2000-microservices-at-wix-986ee382fd1e

[181] B. Silvestre, S. Rossetto, N. Rodriguez, and J. P. Briot. 2010. Flexibility and coordination in event-based, loosely coupled, distributed systems. *Computer Languages, Systems & Structures* 36, 2 (July 2010), 142–157. DOI: https://doi.org/10.1016/j.cl.2009.06.001

[182] Aman Singh. 2025. Effortless CQRS in Dapr with Drasi. Retrieved August 15, 2025 from https://www.linkedin.com/pulse/effortless-cqrs-dapr-drasi-aman-singh-mxsac/

[183] Inc. Stack Exchange. 2023. Stack Exchange Data Dump. Stack Exchange, Inc. Retrieved September 2023 from https://archive.org/details/stackexchange

[184] StackOverflow. 2023. 2023 Developer Survey. StackOverflow Labs. Retrieved September 16, 2024 from https://survey.stackoverflow.co/2023/#overview

[185] StackOverflow. 2023. Stack Overflow—Where Developers Learn, Share, & Build Careers. Stack Exchange Inc. Retrieved July 2023 from http://www.stackoverflow.com/

[186] Apache Flink Statefun. 2023. Stateful Functions: A Platform-Independent Stateful Serverless Stack. Retrieved November 11, 2023 from https://nightlies.apache.org/flink/flink-statefun-docs-master/

[187] Stoffelchen. 2024. ActiveMQ 5.16.1—Messages Get Stuck. Retrieved July 29, 2024 from https://stackoverflow.com/questions/67565608/activemq-5-16-1-messages-get-stuck

[188] Ruoyu Su, Xiaozhou Li, and Davide Taibi. 2024. From microservice to monolith: A multivocal literature review. *Electronics* 13, 8 (2024), 1452.

[189] Johnathon Sullinger. 2024. Microservices + CQRS Implementation. Retrieved July 29, 2024 from https://stackoverflow.com/questions/62142695/microservices-cqrs-implementation

[190] Andrew S. Tanenbaum and Maarten van Steen. 2016. *Distributed Systems: Principles and Paradigms* (2nd ed.). CreateSpace Independent Publishing Platform.

[191] FasterCapital Team. 2025. Event Driven Architecture Best Practices. Retrieved August 10, 2025 from https://fastercapital.com/term/event-driven-architecture-best-practices.html

[192] Temporal. 2025. Durable Execution Solutions. Retrieved July 14, 2025 from https://temporal.io

[193] Branko Terzić, Vladimir Dimitrieski, Slavica Kordić, Gordana Milosavljević, and Ivan Luković. 2018. Development and evaluation of MicroBuilder: A model-driven tool for the specification of REST microservice software architectures. *Enterprise Information Systems* 12, 8–9 (2018), 1034–1057. DOI: https://doi.org/10.1080/17517575.2018.1460766

[194] Carl Thomas. 2024. How to Authenticate Async Event Messaging Between Services in Microservice Architecture. Retrieved July 29, 2024 from https://stackoverflow.com/questions/62109564/how-to-authenticate-async-event-messaging-between-services-in-microservice-archi

[195] Bernardo Toninho, Luís Caires, and Frank Pfenning. 2021. A decade of dependent session types. In *Proceedings of the 23rd International Symposium on Principles and Practice of Declarative Programming (PPDP '21)*. ACM, New York, NY, Article 3, 3 pages. DOI: https://doi.org/10.1145/3479394.3479398

[196] TrueWill. 2024. Ensuring That All Messages Have Been Read from Kafka Topic Using REST Proxy. Retrieved July 29, 2024 from https://stackoverflow.com/questions/57222357/ensuring-that-all-messages-have-been-read-from-kafka-topic-using-rest-proxy

[197] Umbraco. 2024. What Is a Staging Environment? Retrieved June 2024 from https://umbraco.com/knowledge-base/staging-environment/

[198] user2079172. 2024. Long Running REST API with Queues. Retrieved July 29, 2024 from https://stackoverflow.com/questions/33009721/long-running-rest-api-with-queues

[199] user3154653. 2024. Microservice Architecture Dependency. Retrieved July 29, 2024 from https://stackoverflow.com/questions/43378165/microservice-architecture-dependency

[200] Muhammad Usman, Simone Ferlin, Anna Brunstrom, and Javid Taheri. 2022. A survey on observability of distributed edge & container-based microservices. *IEEE Access* 10 (2022), 86904–86919. DOI: https://doi.org/10.1109/ACCESS.2022.3193102

[201] Guilherme Vale, Filipe Figueiredo Correia, Eduardo Martins Guerra, Thatiane de Oliveira Rosa, Jonas Fritzsch, and Justus Bogner. 2022. Designing microservice systems using patterns: An empirical study on quality trade-offs. In *Proceedings of the 2022 IEEE 19th International Conference on Software Architecture (ICSA)*. IEEE, 69–79.

[202] Nicolas Viennot, Mathias Lécuyer, Jonathan Bell, Roxana Geambasu, and Jason Nieh. 2015. Synapse: A microservices architecture for heterogeneous-database web applications. In *Proceedings of the 10th European Conference on Computer Systems (EuroSys '15)*. ACM, New York, NY, Article 21, 16 pages. DOI: https://doi.org/10.1145/2741948.2741975

[203] Malte Viering, Raymond Hu, Patrick Eugster, and Lukasz Ziarek. 2021. A multiparty session typing discipline for fault-tolerant event-driven distributed programming. *Proceedings of the ACM on Programming Languages* 5, OOPSLA, Article 124 (Oct. 2021), 30 pages. DOI: https://doi.org/10.1145/3485501

[204] VMware, Inc. or Its Affiliates. 2023. RabbitMQ—Consumer Acknowledgements and Publisher Confirms. Retrieved July 2023 from https://www.rabbitmq.com/confirms.html

[205] VoiceOfUnreason. 2024. Aggregates in Event Sourcing Pattern. Retrieved July 29, 2024 from https://stackoverflow.com/questions/49985156/aggregates-in-event-sourcing-pattern/49986434#49986434
[206] Pavel Voronin. 2024. What Are the Strategies/Frameworks for Adding New Services to the System Where Integration Is Based on Asynchronous Messages? [Closed]. Retrieved July 29, 2024 from https://stackoverflow.com/questions/56851950/what-are-the-strategies-frameworks-for-adding-new-services-to-the-system-where-i
[207] Wade Waldron. 2023. Idempotency and Ordering in Event-Driven Systems. Retrieved July 2023 from https://www.cockroachlabs.com/blog/idempotency-and-ordering-in-event-driven-systems/
[208] Yingying Wang, Harshavardhan Kadiyala, and Julia Rubin. 2021. Promises and challenges of microservices: An exploratory study. *Empirical Software Engineering* 26, 4 (2021), 63.
[209] Muhammad Waseem, Peng Liang, Aakash Ahmad, Arif Ali Khan, Mojtaba Shahin, Pekka Abrahamsson, Ali Rezaei Nasab, and Tommi Mikkonen. 2023. Understanding the issues, their causes and solutions in microservices systems: An empirical study. arXiv:2302.01894. Retrieved from https://arxiv.org/abs/2302.01894
[210] Muhammad Waseem, Peng Liang, and Mojtaba Shahin. 2020. A systematic mapping study on microservices architecture in DevOPS. *Journal of Systems and Software* 170 (2020), 110798.
[211] Pascal Weisenburger, Johannes Wirth, and Guido Salvaneschi. 2020. A survey of multitier programming. *ACM Computing Surveys* 53, 4, Article 81 (Sept. 2020), 35 pages. DOI: https://doi.org/10.1145/3397495
[212] George Wells. 2005. Coordination languages: Back to the future with Linda. In *Proceedings of the 2nd International Workshop on Coordination and Adaptation Techniques for Software Entities (WCAT '05)*.
[213] Jinfeng Wen, Zhenpeng Chen, Yi Liu, Yiling Lou, Yun Ma, Gang Huang, Xin Jin, and Xuanzhe Liu. 2021. An empirical study on challenges of application development in serverless computing. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '21)*. ACM, New York, NY, 416–428. DOI: https://doi.org/10.1145/3468264.3468558
[214] Laurent Wilfred. 2024. Microservice Architecture Data Sharing/Management. Retrieved July 29, 2024 from https://stackoverflow.com/questions/53100208/microservice-archtecture-data-sharing-management
[215] xm22. 2024. Messaging: How Do Your Messages Look Like. Retrieved July 29, 2024 from https://stackoverflow.com/questions/53025888/messaging-how-do-your-messages-look-like
[216] Xin-Li Yang, David Lo, Xin Xia, Zhi-Yuan Wan, and Jian-Ling Sun. 2016. What security questions do developers ask? A large-scale study of stack overflow posts. *Journal of Computer Science and Technology* 31 (2016), 910–924.
[217] Yang Yang, Zhifeng Chen, Qichao Chu, Haitao Zhang, and George Teo. 2021. Enabling Seamless Kafka Async Queuing with Consumer Proxy. Uber Technologies, Inc. Retrieved from https://www.uber.com/en-SE/blog/kafka-async-queuing-with-consumer-proxy/
[218] K. Youssefi and E. Wong. 1979. Query processing in a relational database management system. In *Proceedings of the 5th International Conference on Very Large Data Bases*, 409–417. DOI: https://doi.org/10.1109/VLDB.1979.718156
[219] He Zhang, Shanshan Li, Zijia Jia, Chenxing Zhong, and Cheng Zhang. 2019. Microservice architecture in reality: An industrial inquiry. In *Proceedings of the 2019 IEEE International Conference on Software Architecture (ICSA)*. IEEE, 51–60.
[220] Xin Zhou, Shanshan Li, Lingli Cao, He Zhang, Zijia Jia, Chenxing Zhong, Zhihao Shan, and Muhammad Ali Babar. 2023. Revisiting the practices and pains of microservice architecture in reality: An industrial inquiry. *Journal of Systems and Software* 195 (2023), 111521. DOI: https://doi.org/10.1016/j.jss.2022.111521
[221] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Wenhai Li, and Dan Ding. 2021. Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study. *IEEE Transactions on Software Engineering* 47, 2 (2021), 243–260. DOI: https://doi.org/10.1109/TSE.2018.2887384
[222] Olaf Zimmermann. 2017. Microservices tenets. *Computer Science—Research and Development* 32, 3–4 (2017), 301–310.
[223] Olaf Zimmermann, Mirko Stocker, Daniel Lubke, Uwe Zdun, and Cesare Pautasso. 2022. *Patterns for API Design: Simplifying Integration with Loosely Coupled Message Exchanges*. Addison-Wesley Professional.
[224] Tilman Zuckmantel, Yongluan Zhou, Boris Düdder, and Thomas Hildebrandt. 2022. Event-based data-centric semantics for consistent data management in microservices. In *Proceedings of the 16th ACM International Conference on Distributed and Event-Based Systems (DEBS '22)*. ACM, New York, NY, 97–102. DOI: https://doi.org/10.1145/3524860.3539807

Received 1 August 2024; revised 12 September 2025; accepted 20 October 2025